Voice for AI Agents and Applications

Vocal Bridge

Voice UI • Concierge Architecture • Client Actions • AI Agent Integration • Voice as a Tool • Voice Evals

Hanwha Life Vietnam

Phiên bản v1.0.0

Ngày 27 tháng 10 năm 2026

Mục lục

1 Giới thiệu khoá học **2**
- 1.1 Vì sao voice . . . 2
- 1.2 Đánh đổi giữa độ trễ và trí tuệ . . . 2
- 1.3 Ba mẫu tích hợp . . . 2
- 1.4 Cấu trúc tài liệu . . . 3
- 1.5 Môi trường thực hành . . . 4

2 Tổng quan Voice UI **5**
- 2.1 Vì sao voice . . . 5
  - 2.1.1 Ví dụ: đặt vé máy bay . . . 5
- 2.2 Voice đang được dùng ở đâu . . . 6
- 2.3 Ba bề mặt tích hợp của Vocal Bridge . . . 6
- 2.4 Production voice stack nếu tự xây . . . 6
- 2.5 Hai kiến trúc và sự đánh đổi . . . 8
- 2.6 Concierge architecture . . . 9
- 2.7 Điểm cần nhớ . . . 10

3 Voice trong ứng dụng **11**
- 3.1 Bài toán: giữ voice và UI đồng bộ . . . 11
- 3.2 Mẫu Client Actions . . . 11
- 3.3 Chuẩn bị . . . 12
- 3.4 Hai artifact của một voice UI agent . . . 12
  - 3.4.1 Prompt và hai luật chịu lực . . . 12
  - 3.4.2 Action schema . . . 12
- 3.5 Tạo agent bằng CLI . . . 13
- 3.6 Mint voice token . . . 14
- 3.7 Chạy thử widget . . . 14
- 3.8 Hook `useAgentActions` . . . 15
- 3.9 Đọc call log . . . 16
- 3.10 Ràng buộc cần giữ . . . 16

4 Voice cho agent có sẵn **17**
- 4.1 Bài toán . . . 17
- 4.2 Cách phổ biến: kiểu “sandwich” . . . 17
- 4.3 Cách của Vocal Bridge . . . 18
- 4.4 Một lượt hội thoại . . . 18
- 4.5 Thực hành: AI Agent Integration mode . . . 19
  - 4.5.1 Chuẩn bị . . . 19
  - 4.5.2 Xây agent dạng text trước . . . 19
  - 4.5.3 Expose agent qua HTTP . . . 20
  - 4.5.4 Prompt của lớp voice . . . 20

4.5.5 Cấu hình `ai-agent.json` . . . 20
4.5.6 Tạo agent . . . 21
4.5.7 Widget và chạy thử . . . 21
4.5.8 Hook `useAIAgent` . . . 22
4.6 Tóm tắt: ba mảnh ghép . . . 22

5 Voice như một tool **23**
5.1 Voice là một function call . . . 23
5.2 Vì sao tự xây không đơn giản . . . 23
5.3 Các tình huống phổ biến . . . 24
5.4 Luồng một cuộc gọi . . . 24
5.5 An toàn khi gọi ra ngoài . . . 25
5.6 Thực hành . . . 25
5.6.1 Chuẩn bị . . . 25
5.6.2 Caller agent: base prompt và purpose . . . 25
5.6.3 Tạo caller agent . . . 25
5.6.4 Tool schema cho Claude . . . 26
5.6.5 Cài đặt tool . . . 27
5.6.6 Để Claude quyết định gọi . . . 27
5.6.7 Tải và nghe lại ghi âm . . . 27
5.7 Tóm tắt . . . 28

6 Đánh giá voice agent **29**
6.1 Eval và Eval-Driven Development . . . 29
6.2 Vì sao voice eval khó hơn text eval . . . 29
6.3 Hai lớp đánh giá . . . 30
6.3.1 Lớp 1 – chỉ số định lượng cứng . . . 30
6.3.2 Lớp 2 – multimodal LLM làm giám khảo . . . 30
6.4 Evaluator `vb eval` . . . 30
6.5 Thực hành . . . 31
6.5.1 Tìm phiên để chấm . . . 31
6.5.2 Chấm một cuộc gọi theo objective . . . 32
6.5.3 Đọc báo cáo . . . 32
6.5.4 Xây một eval suite nhỏ . . . 33
6.5.5 Khép vòng lặp . . . 34
6.6 Ràng buộc cần giữ . . . 35

7 Voice agent trong production **36**
7.1 Bài học từ Docker . . . 36
7.2 Áp vào voice . . . 36
7.3 “Production” nghĩa là gì . . . 36
7.4 Hai bề mặt mới ngoài contact center . . . 37
7.4.1 Voice như một tính năng bên trong ứng dụng . . . 37
7.4.2 Voice cho AI agent . . . 37
7.5 Những khó khăn đội ngũ thực tế hay gặp . . . 38
7.6 Thông điệp chính . . . 38

A Thuật ngữ và tra cứu nhanh **39**
A.1 Bảng thuật ngữ . . . 39
A.1.1 Từ viết tắt và chỉ số . . . 39
A.1.2 Kiến trúc . . . 39
A.1.3 Trải nghiệm hội thoại . . . 40

A.1.4 Phát triển và phương pháp 40
A.2 Lệnh CLI `vb` và SDK 42
A.3 Các helper trong notebook 42

Chương 1

Giới thiệu khoá học

Tài liệu này tổng hợp khoá học *Voice for AI Agents and Applications* của DeepLearning.AI thành một bản đọc liền mạch bằng tiếng Việt. Khoá học được xây dựng cùng **Vocal Bridge** – một công ty trong danh mục đầu tư của AI Fund – do Ashwyn Sharma, CEO của Vocal Bridge, giảng dạy; Andrew Ng giới thiệu mở đầu.

Mục tiêu của khoá học là xây dựng các agent có **giao diện giọng nói** (Voice User Interface – Voice UI): thêm voice vào agent đã có, cho agent dùng voice như một tool (function call), và áp dụng **voice evals** để chấm chất lượng, độ chính xác của agent kèm phản hồi chi tiết để cải thiện.

1.1 Vì sao voice

Phần lớn mọi người thấy nói và nghe dễ hơn nhiều so với viết và đọc. Khi Voice UI ngày càng tin cậy, nó mở ra nhiều ứng dụng mới cho nhiều người hơn. Andrew Ng kể trên bản tin *The Batch* về một trò chơi đố toán chủ đề mèo ông viết cho con gái bảy tuổi: bé vốn chơi bằng bàn phím, nhưng chỉ cần ít công sức với Vocal Bridge là trò chơi có thêm Voice UI – giờ hai bố con hỏi và đáp bằng lời, và trải nghiệm thay đổi hẳn.

1.2 Đánh đổi giữa độ trễ và trí tuệ

Ứng dụng voice từ trước đến nay luôn phải chọn giữa **độ trễ** (latency) và **trí tuệ** (intelligence):

- **Mô hình voice-to-voice thời gian thực** (native voice-in, voice-out) – nhanh, nhưng kém tin cậy và khó kiểm soát.
- **Pipeline** – nhận audio, chuyển giọng nói thành văn bản (Speech-to-Text – STT), cho LLM chạy một agentic workflow, rồi chuyển văn bản thành giọng nói (Text-to-Speech – TTS) để đọc câu trả lời. Tin cậy và kiểm soát được hơn, nhưng cộng dồn độ trễ – vấn đề lớn với hội thoại thời gian thực.

Vocal Bridge giải bài toán này bằng một kiến trúc riêng: một **foreground agent** nhanh lo hội thoại thời gian thực, và một **background agent** lo các workflow cần suy luận, guardrail và tool. Nhờ vậy voice agent có được cả độ trễ thấp lẫn trí tuệ cao. Kiến trúc này được trình bày ở mục 2.6.

1.3 Ba mẫu tích hợp

Khoá học dạy ba mẫu tích hợp (integration pattern), "gặp người học ở đúng chỗ họ đang đứng":

1. **Voice nhúng trong ứng dụng** – ví dụ một trò chơi hay công cụ năng suất mà người dùng vừa ra lệnh bằng giọng nói, vừa click trực tiếp trên UI. Voice agent phải làm UI thay đổi khi người dùng nói, *và* phải biết khi người dùng click vào thứ gì đó. Chính sự nhận biết hai chiều này làm trải nghiệm tự nhiên (Chương 3).

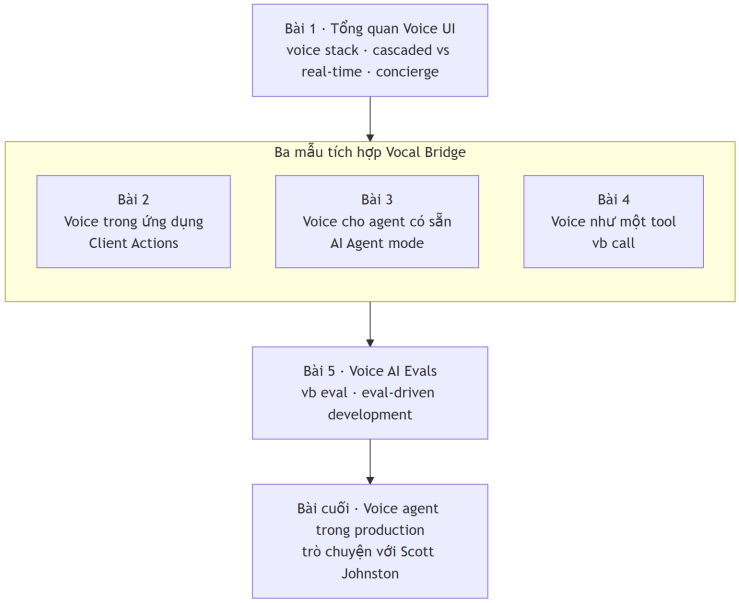
2. **Voice cho agent có sẵn** – bạn đã có agent chạy bằng Claude, GPT hay LangChain với logic riêng và không muốn viết lại. Vocal Bridge đứng trước agent như một lớp mỏng: chuyển giọng nói thành ý định, và tự quyết định khi nào chuyển câu hỏi cho agent của bạn, khi nào tự xử lý các câu xã giao (Chương 4).

3. **Voice như một tool LLM có thể gọi** – một agent tuyển dụng điều phối lịch phỏng vấn: ứng viên nhắn tin báo lịch rảnh, agent nhắn lại xác nhận rồi *gọi điện* để chốt ngày phù hợp nhất. Hoặc một agent brainstorm đang chat với bạn chủ động đề nghị "nói chuyện trực tiếp sẽ nhanh hơn" và mở một phiên voice. Agent tự chọn phương thức (modality) hợp với từng thời điểm (Chương 5).

Cuối khoá có cuộc trò chuyện với Scott Johnston – cựu CEO Docker, thành viên hội đồng quản trị Vocal Bridge – về bối cảnh voice agent và những gì cần để đưa voice vào production (Chương 7).

1.4 Cấu trúc tài liệu

Mỗi bài của khoá học tương ứng một chương. Bài 1 trình bày stack voice truyền thống để hiểu Vocal Bridge trừu tượng hoá những gì, rồi demo cả ba mẫu tích hợp; các bài 2–5 có notebook thực hành.



Hình 1.1: Lộ trình khoá học – mỗi khối tương ứng một chương của tài liệu

Chương	Bài / notebook	Nội dung chính
2	Bài 1 (không có notebook)	Vì sao voice, voice được dùng ở đâu, production voice stack, cascaded và real-time, concierge architecture.
3	Bài 2 – `L2.ipynb`	Client Actions: kênh hai chiều giữa agent và UI; trò chơi tic-tac-toe điều khiển bằng giọng nói và chuột.
4	Bài 3 – `L3.ipynb`	AI Agent mode: bọc voice quanh agent Claude + web search có sẵn qua `useAIAgent`.
5	Bài 4 – `L4.ipynb`	Cho Claude tool `make_phone_call` để tự quyết định gọi điện thật qua `vb call`.
6	Bài 5 – `L5.ipynb`	Eval-driven development với evaluator đa phương thức `vb eval`.
7	Bài cuối	Từ demo đến production: bài học từ Docker, hai bề mặt mới của voice, các khó khăn thường gặp.
A	–	Bảng thuật ngữ, lệnh CLI `vb`, các helper trong notebook.

Bảng 1.1: Cấu trúc tài liệu

1.5 Môi trường thực hành

Trong môi trường của DeepLearning.AI mọi thứ đã được chuẩn bị sẵn: tài khoản Vocal Bridge, API key nằm trong biến môi trường, CLI `vb` đã xác thực – chỉ cần chạy các cell. Muốn chạy trên máy riêng thì:

1. Vào dashboard Vocal Bridge, tạo API key.
2. Đặt key vào biến môi trường `VOCAL_BRIDGE_API_KEY` (thường ghi trong `.env`). Bài 3 và 4 cần thêm `ANTHROPIC_API_KEY`.
3. Cài CLI của Vocal Bridge bằng `pip`, rồi đăng nhập bằng `vb auth login`.

Mỗi notebook đi kèm `helpers.py` gói phần boilerplate (gọi REST API, gọi CLI, render widget React trong notebook) để bài học tập trung vào khái niệm của Vocal Bridge. Danh sách đầy đủ các helper ở Phụ lục A.3; các lệnh CLI dùng trong khoá học ở Phụ lục A.2.

Khoá học có sự đóng góp của Eli Chen, Rakesh Utekar (AI Fund), Aditi Dhar, Jitesh Gupta (Vocal Bridge), Brendan Brown và Esmaeil Gargari (DeepLearning.AI).

Chương 2

Tổng quan Voice UI

Bài này không có notebook. Mục tiêu là xây một mô hình tư duy rõ ràng về ba điều: một voice agent production thực sự gồm những gì, voice nằm ở đâu trong stack của bạn, và những gì sẽ được xây bằng tay từ Chương 3. Chương đi từ lý do chọn voice, bức tranh ứng dụng, stack production nếu tự xây, đến hai kiến trúc phổ biến và câu trả lời của Vocal Bridge – **concierge architecture**.

2.1 Vì sao voice

Có bốn lý do để coi voice là một phương thức tương tác hạng nhất (first-class modality) trong kỷ nguyên AI:

1. **Tự nhiên nhất** – con người đã nói chuyện hàng trăm nghìn năm, còn gõ phím mới có vài thập kỷ. Voice là cách diễn đạt ý định ít ma sát nhất: không menu, không form, không cuộn trang.
2. **Paralinguistics** (cận ngôn ngữ) – điểm hay bị đánh giá thấp. Một câu nói mang nhiều tín hiệu hơn hẳn cùng câu đó dưới dạng chữ: tone, nhịp, sự ngập ngừng, độ khẩn cấp, ngữ điệu (prosody), cảm xúc. Agent nghe được bạn nói *như thế nào*, chứ không chỉ nói *cái gì*, sẽ hành xử rất khác agent chỉ đọc văn bản.
3. **Đa phương thức** (multimodality) – voice không thay thế UI mà đi cùng UI: nói ra ý định, nhìn lại thông tin có cấu trúc. Voice agent tốt nhất dùng voice và GUI cùng lúc, mỗi phương thức làm phần việc nó giỏi nhất.
4. **Khả năng tiếp cận** (accessibility) – voice tiếp cận được người không gõ phím được, không đọc được màn hình, không thao tác được GUI phức tạp. Thường đây là giao diện dễ tiếp cận nhất mà bạn có thể phát hành.

Với AI thời gian thực, voice cuối cùng đã dùng được như một giao diện; kết hợp với UI sẵn có, nó tạo ra bước nhảy về cảm giác sử dụng sản phẩm.

2.1.1 Ví dụ: đặt vé máy bay

Tiêu chí	GUI truyền thống	Voice
Thao tác	Bốn pha: mở app; tìm kiếm, chọn ngày, lọc; chọn phương án; thanh toán và xác nhận. Mỗi pha lại có nhiều bước con.	Một câu: "Book me a flight from SF to NYC next Friday morning." Agent lo phần màn hình, bạn chỉ cần xác nhận bằng lời.
Thời gian	Dễ dàng hơn hai phút, chạm liên tục.	Dưới 10 giây, vẫn ở trong cuộc hội thoại.

Bảng 2.1: Cùng một tác vụ đặt vé máy bay qua GUI và qua voice

Điểm mấu chốt không phải voice thay GUI, mà là voice **xoá ma sát giữa ý định và kết quả**. Mẫu này sẽ lặp lại trong mọi demo của khoá học.

2.2 Voice đang được dùng ở đâu

Nếu bạn từng nghe về voice AI vài năm gần đây, bối cảnh gần như chắc chắn là **contact center**: hỗ trợ khách hàng, tổng đài tự động (Interactive Voice Response – IVR), Agent Assist, Call QA. Ngành bắt đầu từ đây vì kinh tế rõ ràng, workflow bị ràng buộc chặt, và có sẵn người mua.

Nhưng đó chỉ là một lát mỏng của cái mà khoá học gọi là "vũ trụ voice agent". Phần lớn hơn là voice như một *tính năng* của mọi loại phần mềm, ở mọi ngành:

Lĩnh vực	Ví dụ sử dụng voice
Contact center	Support, IVR, Agent Assist, Call QA – nơi ngành bắt đầu.
Fintech	Voice làm giao diện tới tài khoản, danh mục đầu tư.
Trên xe hơi	Không thể rời tay khỏi vô lăng.
Y tế	Đọc chép lâm sàng (clinical dictation), tiếp nhận bệnh nhân, check-in.
Devtools	Trợ lý lập trình có thể trò chuyện.
Accessibility	Người dùng khó thao tác GUI.
Năng suất	Agent cho cuộc họp, lịch, hộp thư.
Giáo dục	Trẻ em chưa biết đọc.
Khác	Gaming, dịch vụ hiện trường (field service), … danh sách vẫn đang dài thêm.

Bảng 2.2: Các bối cảnh sử dụng voice agent

Khung nhìn của cả khoá học: *voice hôm nay sống ở contact center; voice ngày mai sống ở mọi nơi lập trình viên phát hành phần mềm*.

2.3 Ba bề mặt tích hợp của Vocal Bridge

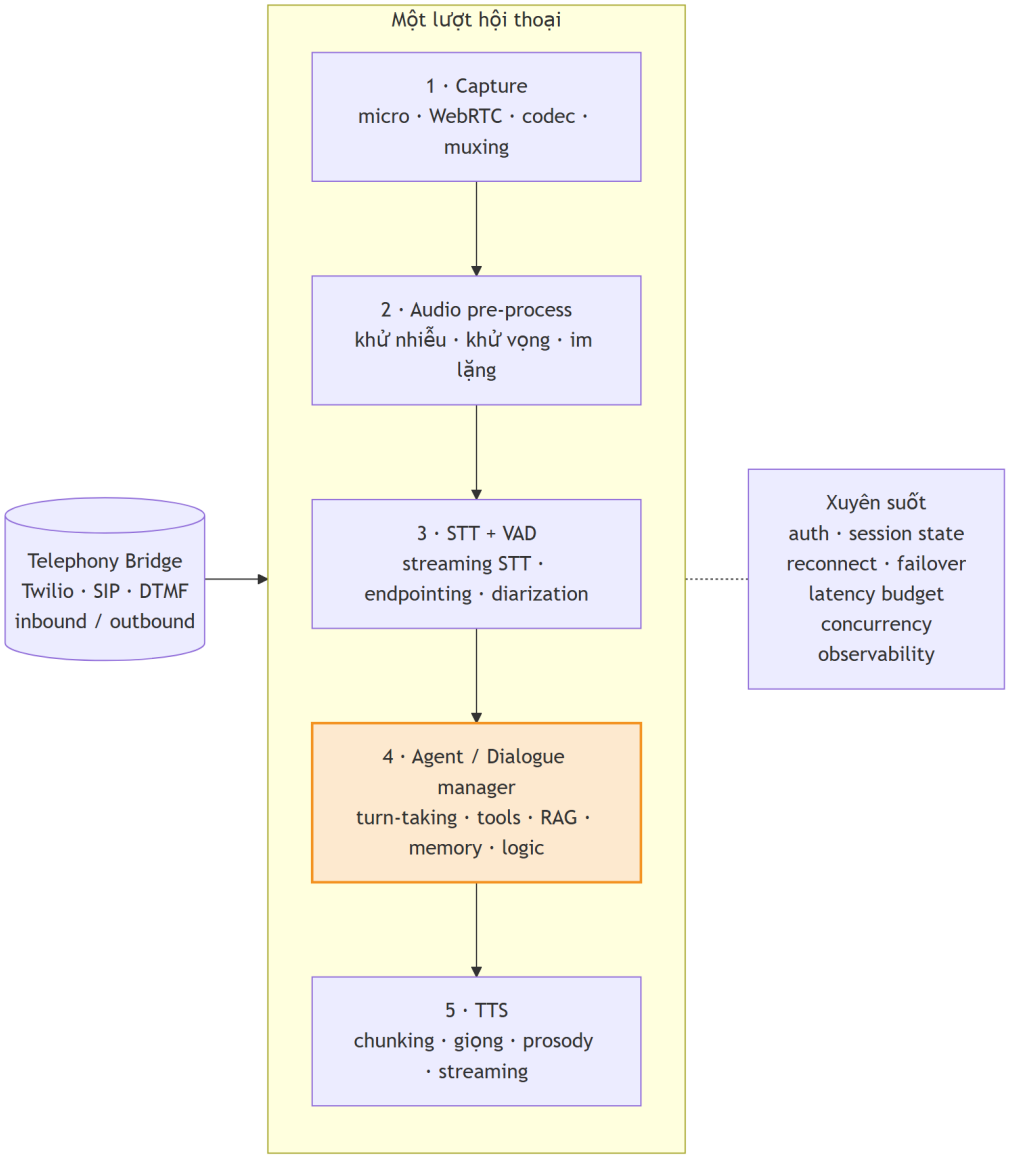
Phần còn lại của khoá học làm việc với ba bề mặt (surface) mà Vocal Bridge cắm vào stack của bạn:

Bề mặt	Công cụ	Cách dùng
Voice cho ứng dụng	React SDK	Thêm component `VocalBridgeProvider` để voice hoá bất kỳ ứng dụng nào; agent kích hoạt action và UI phản ứng (Chương 3).
Voice cho agent	React SDK	Hook `useAIAgent` cho LLM agent có sẵn một giọng nói chỉ với hai dòng code (Chương 4).
Voice như một tool	CLI	Lệnh `vb call` cho agent gọi điện thoại thật; voice trở thành một function mà LLM có thể gọi (Chương 5).

Bảng 2.3: Ba bề mặt tích hợp của Vocal Bridge

2.4 Production voice stack nếu tự xây

Để thấy Vocal Bridge gom lại những gì, hãy nhìn vào thứ phải tự xây nếu tự nối mọi thứ. Đây là stack production đầy đủ (Hình 2.1).



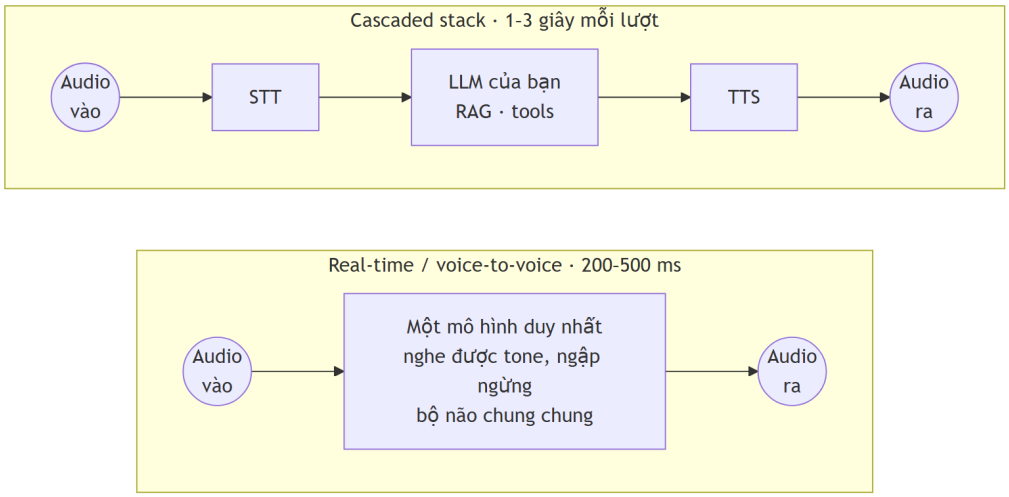
Hình 2.1: Production voice stack – chỉ khối Agent là sản phẩm của bạn, phần còn lại là “hệ thống ống nước”

Khối	Trách nhiệm
Capture	Xin quyền microphone, nhận audio qua WebRTC, codec, ghép track (track muxing), đổi thiết bị khi đang chạy (device hot swap) – chỉ để đưa được audio vào hệ thống.
Audio pre-process	Khử nhiễu (noise cancellation), khử tiếng vọng (echo cancellation), xử lý khung im lặng. Không hào nhoáng, nhưng quyết định agent có nghe rõ hay không.
STT + VAD	Streaming STT (Whisper, Deepgram…) vì agent thời gian thực cần văn bản dạng luồng; phát hiện giọng nói (Voice Activity Detection – VAD) để biết khi nào có người nói; endpointing để biết người dùng đã nói xong lượt; diarization nếu có nhiều người nói.
Agent / Dialogue manager	Turn-taking, định tuyến tool, RAG pipeline, memory, prompt, business logic. Đây là khối *duy nhất* thực sự là sản phẩm của bạn.
TTS	Chia câu (sentence chunking), chọn giọng, cấu hình prosody, tổng hợp dạng streaming, chọn giữa nhiều nhà cung cấp TTS.
Telephony Bridge	Nhánh nằm trên Capture: làm việc với nhà cung cấp như Twilio, giao thức SIP, tín hiệu bàn phím DTMF, xử lý inbound và outbound riêng.
Xuyên suốt	Xác thực, quản lý trạng thái phiên, kết nối lại, failover, ngân sách độ trễ (latency budget), concurrency, observability.

Bảng 2.4: Các khối của production voice stack

2.5 Hai kiến trúc và sự đánh đổi

Voice AI hiện có hai kiến trúc chính, và giữa chúng là một sự đánh đổi không ai muốn chấp nhận (Hình 2.2).



Hình 2.2: Cascaded stack và real-time stack

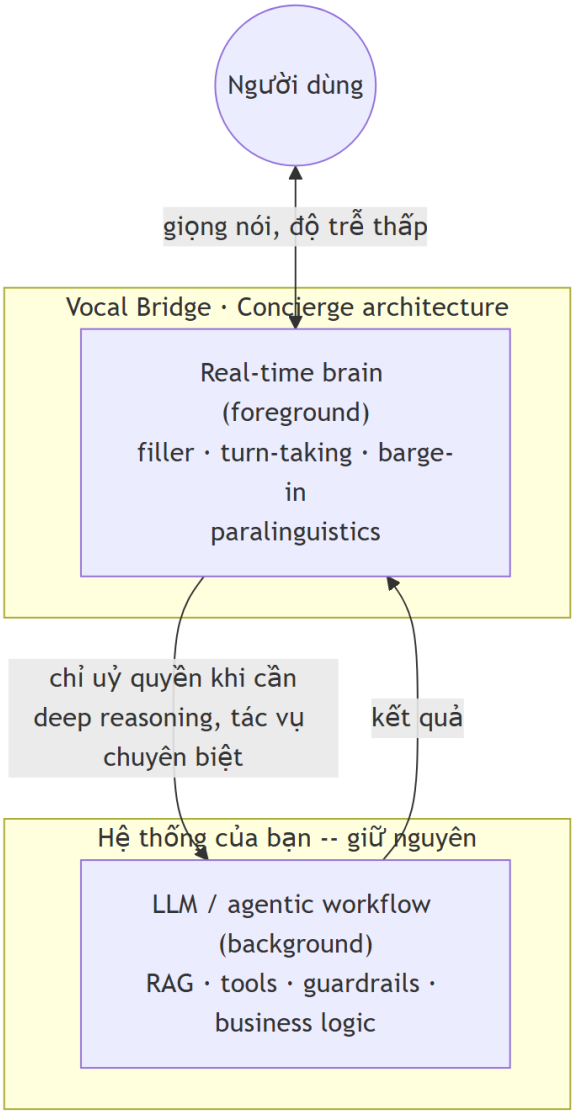
Tiêu chí	Cascaded (STT → LLM → TTS)	Real-time (voice-to-voice)
Thành phần	Ba mô hình nối tiếp.	Một mô hình duy nhất, vào là giọng nói, ra là giọng nói.
Độ trễ	1–3 giây mỗi lượt, đầu cuối – không lý tưởng cho hội thoại thời gian thực.	200–500 ms – đúng mức cần cho voice agent.
Paralinguistics	Mất hết: STT loại bỏ tone, cảm xúc, nhịp trước khi agent nhìn thấy đầu vào.	Giữ được: mô hình nghe tone, cao độ, ngập ngừng, cảm xúc.
Suy luận	Dùng nguyên LLM stack của bạn, suy luận sâu được.	"Bộ não" chung chung; nối với RAG, tool, tri thức chuyên ngành không đơn giản.
Gỡ lỗi	Dễ, mọi bước đều là văn bản.	Khó kiểm soát hơn.

Bảng 2.5: So sánh hai kiến trúc voice AI

Vậy là phải chọn: *độ trễ thấp* hoặc *suy luận sâu*; *sự tự nhiên* hoặc *tận dụng bộ não sẵn có*.

2.6 Concierge architecture

Câu trả lời của Vocal Bridge là **concierge architecture**: một bộ não thời gian thực lo cuộc hội thoại – câu đệm (filler), turn-taking, paralinguistics – và *chỉ* uỷ quyền (delegate) cho LLM của bạn khi cần suy luận sâu hoặc khi cần một tác vụ chuyên biệt mà LLM hay agentic workflow của bạn làm tốt. Kết quả: độ trễ thời gian thực, và LLM stack của bạn giữ nguyên.



Hình 2.3: Concierge architecture – foreground lo hội thoại, background lo suy luận

Cách hiện thực cụ thể là "bí quyết riêng" của Vocal Bridge; hình dạng tổng quát như Hình 2.3 là đủ để bước vào các chương thực hành.

2.7 Điểm cần nhớ

- Voice xoá ma sát giữa ý định và kết quả; nó đi cùng GUI chứ không thay GUI.
- Contact center chỉ là điểm khởi đầu; voice sẽ là tính năng của mọi phần mềm.
- Trong production voice stack, chỉ khối agent là sản phẩm của bạn; phần còn lại là hạ tầng mà nền tảng nên lo.
- Cascaded thì thông minh nhưng chậm và mất paralinguistics; real-time thì nhanh nhưng bộ não chung chung. Concierge architecture kết hợp ưu điểm của cả hai.

Chương 3

Voice trong ứng dụng

Phần lớn ứng dụng voice bắt người dùng chọn giữa *nói* và *click*, trong khi ứng dụng thật cần cả hai. Chương này (bài 2, notebook `L2.ipynb`) xây một trò chơi tic-tac-toe điều khiển bằng giọng nói, trong đó lệnh nói và cú click chuột đi chung một kênh duy nhất. Trò chơi chỉ là cái cớ; thứ cần học là **Client Actions** – kênh hai chiều giúp voice agent và UI chuyển động như một hệ thống.

3.1 Bài toán: giữ voice và UI đồng bộ

Voice agent độc lập – chỉ nghe và nói – là trường hợp dễ. Trường hợp khó, và cũng là thứ hầu hết mọi người muốn phát hành, là voice sống *bên trong* một sản phẩm: dashboard, trò chơi, luồng thanh toán. Khi đó agent và UI phải khớp nhau trong suốt tương tác, không chỉ lúc bắt đầu. Ba vòng lặp diễn ra liên tục:

1. Người dùng **nói** – UI phải cập nhật theo yêu cầu.
2. Người dùng **click** – agent phải biết điều đó ngay giữa cuộc hội thoại.
3. **Trạng thái** đổi ở bất kỳ phía nào – phía kia phải biết *ngay*, không phải ở lượt sau.

Nếu ba vòng lặp này không nhanh và không hai chiều, voice và UI trôi khỏi nhau và trải nghiệm đổ vỡ.

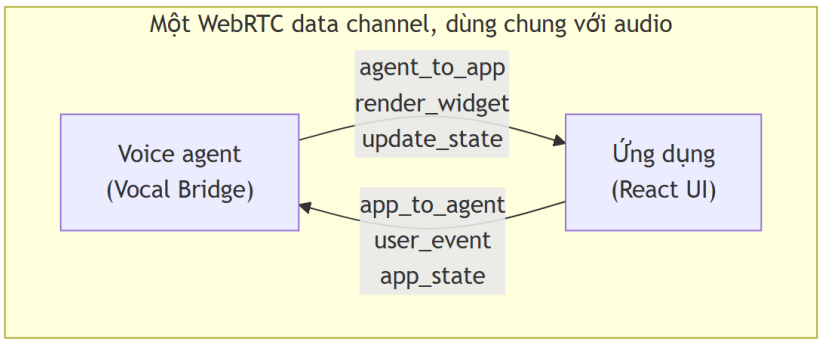
3.2 Mẫu Client Actions

Client Actions là một kênh thông điệp có kiểu (typed message channel) duy nhất giữa voice agent và ứng dụng, mở suốt phiên. Phía nào cũng gửi được, phía kia phản ứng ngay.

Chiều	Thông điệp	Ý nghĩa
Agent → App	`render_widget`	Bảo UI hiện hoặc thay đổi một thứ gì đó.
	`update_state`	Đẩy trạng thái phía agent sang ứng dụng.
App → Agent	`user_event`	Người dùng click, gõ, kéo thả.
	`app_state`	Trạng thái hiện tại của UI, gửi về làm ngữ cảnh.

Bảng 3.1: Hai chiều của Client Actions (dạng tổng quát)

Chi tiết quan trọng: cả hai chiều đi trên **cùng WebRTC data channel với luồng audio**. Agent và ứng dụng thấy nhau như một hệ thống, không phải hai service "nói chuyện bắc cầu" qua HTTP.



Hình 3.1: Client Actions – một kênh, hai chiều, dùng chung với audio

Hình dạng này – hai chiều, một kênh, ngay trong lúc hội thoại – là khuôn mẫu cho mọi tình huống "voice trong ứng dụng" sau này; các biến thể khác chỉ là thay tên thông điệp.

3.3 Chuẩn bị

```
import json
import os
from pathlib import Path

from helpers import load_env, mint_token, vb, append_to_env, voice_widget

load_env()
assert os.getenv("VOCAL_BRIDGE_API_KEY"), "Set VOCAL_BRIDGE_API_KEY in .env"
print("VB API key loaded")
```

Ba helper chính của bài: `mint_token` lấy voice session token, `vb` bọc CLI, và `voice_widget` render widget React ngay trong notebook (Phụ lục A.3).

3.4 Hai artifact của một voice UI agent

Mỗi voice UI agent đi kèm hai file: **prompt** – "bộ não" gồm tính cách và luật chơi; và **action schema** – khai báo thông điệp nào được phép đi giữa agent và UI.

3.4.1 Prompt và hai luật chịu lực

Đầu prompt (`agents/voice-app/prompt.md`) có hai luật cứng, giữ cho voice và UI khoá chặt với nhau:

- **RULE A** – luôn kết thúc phản hồi bằng *lời nói*, không bao giờ bằng một tool call. Nếu agent kết thúc bằng tool call, người dùng chỉ nghe thấy im lặng và demo "chết".
- **RULE B** – *client sở hữu bàn cờ*, agent thì không. Sau mỗi nước đi, client gửi sự kiện `board_sync` kèm toàn bộ bàn cờ; agent coi đó là sự thật gốc (ground truth).

3.4.2 Action schema

```
actions = json.loads(Path("agents/voice-app/actions.json").read_text())
for a in actions:
    arrow = "→" if a["direction"] == "agent_to_app" else "←"
    print(f"  {arrow} {a['name']:<22} {a['description']}")
```

Action	Chiều	Vai trò trong trò chơi
`show_tic_tac_toe`	`agent_to_app`	Hiện bàn cờ mới khi người dùng muốn chơi.
`place_mark`	`agent_to_app`	Agent đặt dấu O của mình (hàng, cột, người chơi).
`user_move`	`agent_to_app`	Agent đặt dấu X thay người dùng khi họ nói nước đi (ví dụ "Center" → `{row:1, col:1}`).
`clear_ui`	`agent_to_app`	Xoá UI.
`user_placed_mark`	`app_to_agent`	Người dùng click một ô; gửi kèm vị trí, toàn bộ bàn cờ, trạng thái, lượt.
`board_sync`	`app_to_agent`	Gửi ngầm sau mỗi nước đi để agent luôn khớp với màn hình.

Bảng 3.2: Action schema của trò chơi tic-tac-toe

3.5 Tạo agent bằng CLI

Một lệnh CLI tạo agent mới từ hai file trên:

```
# Idempotent: if an agent ID is already set, reuse it. Otherwise create one.
agent_id = os.environ.get("VOCAL_BRIDGE_AGENT_ID_L2", "").strip()
if agent_id:
    print(f"Using pre-provisioned agent: {agent_id}")
else:
    result = vb(
        "agent", "create",
        "--name", "lesson-2-voice-app",
        "--style", "Chatty",
        "--prompt-file", "agents/voice-app/prompt.md",
        "--client-actions-file", "agents/voice-app/actions.json",
        "--deploy-targets", "web",
        json_output=True,
    )
    agent = result["agent"]
    agent_id = agent["id"]
    append_to_env("VOCAL_BRIDGE_AGENT_ID_L2", agent_id)
```

Tham số	Ý nghĩa
`--style Chatty`	Chọn backend OpenAI Realtime để giữ hội thoại; mặc định tốt sẵn.
`--prompt-file`	File prompt của agent.
`--client-actions-file`	Trỏ tới action schema – chính tham số này nối kênh hai chiều. Thiếu nó, có voice nhưng không có tương tác UI.
`--deploy-targets`	`web`: truy cập từ trình duyệt qua WebRTC, không cấp số điện thoại; `phone`: agent có số điện thoại riêng (Chương 5); `both`: cả web và telephony, hữu ích cho agent lai.

Bảng 3.3: Tham số của `vb agent create` trong bài 2

Vì sao viết idempotent: nếu ID agent đã có trong môi trường (sandbox cấp sẵn, hoặc đã chạy cell này trước đó) thì dùng lại; nếu chưa thì tạo mới và lưu ID vào `.env`. Nhờ vậy chạy lại cell không sinh thêm agent mới mỗi lần.

3.6 Mint voice token

Để trình duyệt kết nối tới agent, backend *mint* một **voice session token** ngắn hạn bằng cách POST tới `/api/v1/token` kèm API key. Trình duyệt dùng token đó để mở phiên – **API key không bao giờ rời khỏi server**. Trong ứng dụng thật, cả route backend chỉ khoảng mười dòng:

```
import requests

def mint_token(agent_id, api_key):
    res = requests.post(
        "https://vocalbridgeai.com/api/v1/token",
        headers={
            "X-API-Key": api_key,
            "X-Agent-Id": agent_id,
            "Content-Type": "application/json",
        },
        json={"participant_name": "Web User"},
    )
    return res.json()  # → { token, livekit_url, room_name, ... }
```

Gọi helper trong notebook in ra ba thông tin: `session_name`; `agent_mode` – ở đây là `openai_concierge`, nghĩa là dùng mô hình OpenAI Realtime phía sau cùng concierge architecture của Vocal Bridge; và `token` (in rút gọn) dùng để xác thực phiên.

```
token = mint_token(agent_id, participant_name="Notebook Learner")
print(f"session_name:   {token.session_name}")
print(f"agent_mode:     {token.agent_mode}")
print(f"token (truncd): {token.token[:40]}…")
```

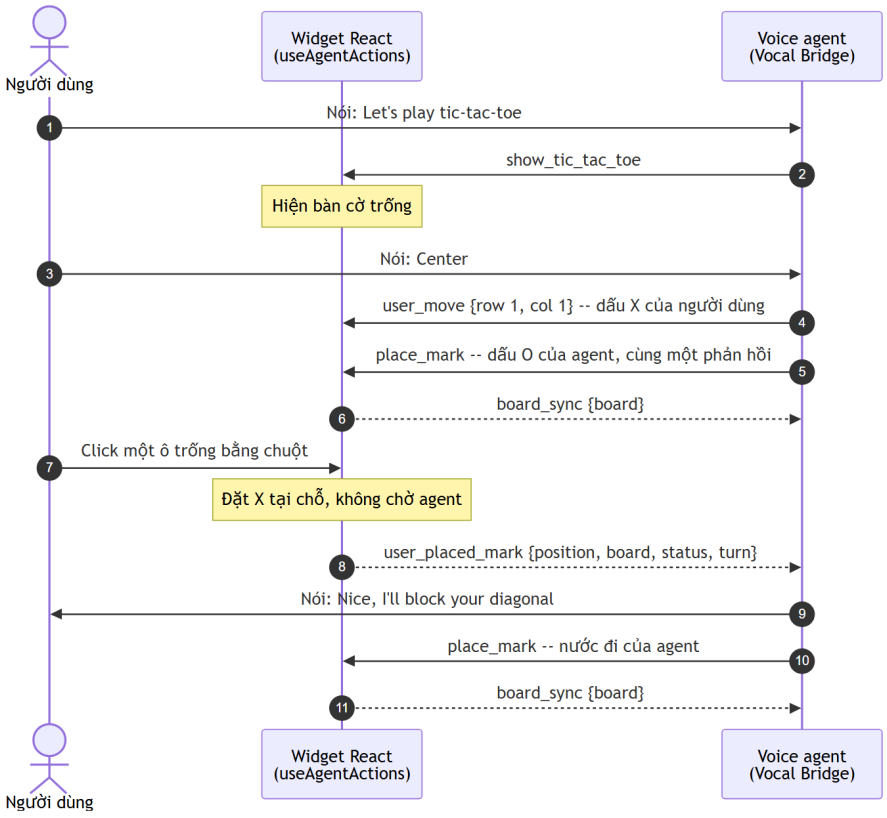
3.7 Chạy thử widget

```
from IPython.display import HTML
HTML(voice_widget(token))
```

Bấm **Connect**, cho phép microphone, rồi thử:

1. Nói "Let's play tic-tac-toe." – agent kích hoạt `show_tic_tac_toe`, bàn cờ xuất hiện.
2. Nói "Center." – agent kích hoạt `user_move` (dấu X của bạn) *và* `place_mark` (dấu O của nó) trong cùng một phản hồi.
3. **Click** một ô trống – widget đặt X ngay tại chỗ và gửi `user_placed_mark` kèm bàn cờ mới; agent đáp lại bằng `place_mark`.

Sau mỗi nước đi (của bạn hay của agent), widget lặng lẽ gửi `board_sync` để mô hình trong đầu agent luôn khớp với thứ đang hiện trên màn hình (Hình 3.2).



Hình 3.2: Sequence diagram – một ván tic-tac-toe kết hợp giọng nói và chuột

Đoạn hội thoại trong video demo:

```
User:  Let's play tic-tac-toe.
Agent: You're X, your move.
User:  Can I take the center, please?
Agent: Corner, better view.
Agent: Blocking that.
User:  I will take bottom left.
Agent: Okay, you got me. Nicely played.
```

3.8 Hook `useAgentActions`

Toàn bộ widget xoay quanh **một React hook** – `useAgentActions` – cho cả hai chiều của Client Actions trên một kênh: `onAction` lắng nghe sự kiện agent bắn sang, còn `sendAction` gửi sự kiện từ ứng dụng về agent.

```
import { useAgentActions } from '@vocalbridgeai/react';

function TicTacToe() {
  const { onAction, sendAction } = useAgentActions();

  // Listen for actions FROM the agent
  useEffect(() => {
    return onAction('show_tic_tac_toe', (payload) => {
      setBoard(Array(9).fill(''));
      setBoardVisible(true);
    });
```

```
  }, [onAction]);

  // Send actions TO the agent
  const handleClick = (i) => {
    sendAction('user_placed_mark', {
      position: i, board: nextBoard, status, turn, ...
    });
  };
}
```

Cùng một hook, hai chiều, trên cùng WebRTC data channel. Theo giảng viên, đây là phần riêng có của Vocal Bridge: phần lớn nền tảng voice bắt bạn chọn giữa nói và phản ứng trên UI, còn ở đây cả hai là một kênh.

3.9 Đọc call log

Khi kết thúc phiên, Vocal Bridge lưu một log có cấu trúc gồm mọi lượt nói, mọi action được kích hoạt và mọi sự kiện gửi về. Lấy log mới nhất chỉ cần một lệnh CLI:

```
vb("agent", "use", agent_id)
logs = vb("logs", "--limit", "1", json_output=True)
if not logs:
    print("No calls yet — try the widget above first.")
else:
    print(vb("logs", "show", logs['sessions'][0]['id']))
```

Đây là điểm móc vào cho analytics, gỡ lỗi và evals – chủ đề của Chương 6.

3.10 Ràng buộc cần giữ

- Agent luôn kết thúc lượt bằng lời nói, không bằng tool call (RULE A).
- UI là nguồn sự thật cho trạng thái; agent đọc trạng thái từ `board_sync` / `app_state` thay vì tự suy đoán (RULE B).
- Gửi đồng bộ trạng thái sau *mọi* thay đổi, kể cả thay đổi do agent gây ra.
- Action schema phải được truyền khi tạo agent (`--client-actions-file`), nếu không kênh hai chiều không tồn tại.
- API key chỉ nằm ở backend; trình duyệt chỉ cầm voice session token ngắn hạn.

Chương 4

Voice cho agent có sẵn

Chương này (bài 3, notebook `L3.ipynb`) thêm voice cho một agent *đã có* chỉ với khoảng mười dòng code, không đụng vào logic sẵn có: prompt, RAG, tool đều giữ nguyên. Mẫu tích hợp được dùng là **AI Agent Integration mode** của Vocal Bridge.

4.1 Bài toán

Tình huống quen thuộc: bạn đã có một LLM agent trong production – prompt đã tinh chỉnh, tool chạy tốt, RAG đã nối, eval suite toàn màu xanh. Rồi ai đó hỏi: "Thêm giao diện voice được không?". Điều cuối cùng bạn muốn là viết lại tất cả. Điều bạn thực sự muốn:

1. Giữ nguyên agent: prompt, tool, RAG, evals.
2. Thêm voice như một **lớp vận chuyển** (transport layer), không phải xây lại.
3. Không phải hy sinh độ trễ hay sự tự nhiên để có được điều đó.

Câu hỏi của chương: *làm sao bọc voice quanh một agent mà không phá cách nó đang hoạt động?*

4.2 Cách phổ biến: kiểu "sandwich"

Phần lớn nền tảng làm theo kiểu **sandwich**: STT ở trước, agent ở giữa, TTS ở sau. Trông đơn giản – cho đến khi nhận ra agent vừa bị giao thêm việc gì. Agent giờ phải sở hữu luồng hội thoại: turn-taking, barge-in (người dùng chen ngang), paralinguistics; trong khi STT đã vứt bỏ tone, cao độ, ngập ngừng, cảm xúc trước khi agent kịp thấy đầu vào. Cái giá thực sự:

- **Độ trễ tăng** – mỗi lượt trả tuần tự STT + LLM + TTS.
- **Agent làm việc không được thiết kế để làm** – backchannel ("mm-hmm"), khi nào chờ, khi nào hỏi lại.
- **Mất hoàn toàn tín hiệu paralinguistic**.
- **Người dùng ngồi trong im lặng** khi agent chạy, vì không thành phần nào trong stack giữ cho cuộc hội thoại sống.

Sandwich đủ cho demo, nhưng không sống sót khi gặp người dùng thật.

4.3 Cách của Vocal Bridge

Vocal Bridge đặt một **bộ não voice-to-voice thời gian thực** trước agent của bạn. Bộ não này có mô hình riêng, lo toàn bộ phần việc hội thoại – lắng nghe, turn-taking, paralinguistics, "chất keo" hội thoại. Khi người dùng hỏi điều gì thực chất, nó *chỉ uỷ quyền đúng câu hỏi đó* cho agent của bạn. Agent giữ nguyên.

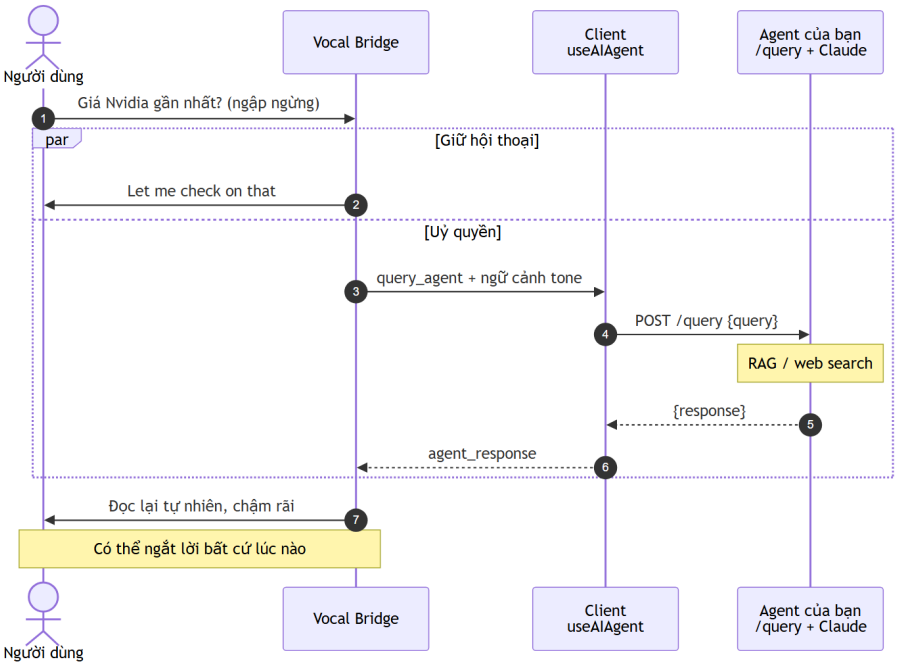
Tiêu chí	Sandwich	Vocal Bridge (AI Agent mode)
Độ trễ	Cao: thác nước STT → LLM → TTS cho mọi lượt.	Thấp: phần hội thoại không đi qua thác nước đó.
Paralinguistics	Mất ở bước STT.	Giữ lại: bộ não nghe tone, ngập ngừng và đóng gói thành ngữ cảnh khi uỷ quyền.
Khi agent đang chạy	Người dùng chờ trong im lặng.	Bộ não giữ hội thoại; người dùng vẫn nói tiếp được.
Công sức với agent	Agent phải gánh logic hội thoại.	Không viết lại: agent trả lời truy vấn có cấu trúc như với chat.

Bảng 4.1: Sandwich so với concierge architecture khi bọc voice quanh agent

4.4 Một lượt hội thoại

Người dùng hỏi "What's the most recent Nvidia stock price?" với giọng hơi ngập ngừng. Bộ não Vocal Bridge nghe được cả câu hỏi lẫn sự không chắc chắn. Nó *lập tức* nói một câu ghi nhận ngắn để người dùng không rơi vào khoảng lặng chết ("Let me check on that"), *đồng thời* gửi cho agent một truy vấn có cấu trúc (`query_agent`) kèm thông tin về tone và các tín hiệu paralinguistic khác.

Agent chạy đúng code path như với chatbot – RAG trên knowledge graph nội bộ, hoặc tool web search – không thêm chút logic hội thoại nào. Bộ não Vocal Bridge nhận câu trả lời và đọc lại một cách tự nhiên: chậm hơn, có ngắt nghỉ để xác nhận vì người dùng nghe có vẻ không chắc. Người dùng có thể ngắt lời bất cứ lúc nào.



Hình 4.1: Sequence diagram – một lượt uỷ quyền câu hỏi trong AI Agent mode

Voice ở bên ngoài, agent không bị đụng tới ở giữa – đó là toàn bộ mẫu tích hợp.

4.5 Thực hành: AI Agent Integration mode

Luồng xử lý trong notebook:

1. Người dùng nói → Vocal Bridge chuyển thành văn bản.
2. Vocal Bridge bắn `query_agent` qua data channel.
3. Endpoint của bạn chạy Claude (hoặc GPT, Gemini, mô hình nội bộ).
4. Bạn trả về một chuỗi → Vocal Bridge nói chuỗi đó.

4.5.1 Chuẩn bị

Bài này cần cả `VOCAL_BRIDGE_API_KEY` và `ANTHROPIC_API_KEY`. Ngoài `mint_token`, `vb`, `voice_widget` quen thuộc, có thêm `start_query_server` / `stop_query_server` – dựng một server FastAPI nhỏ trong thread nền.

```
from helpers import (
    load_env, mint_token, vb, append_to_env, voice_widget,
    start_query_server, stop_query_server,
)

load_env()
assert os.getenv("VOCAL_BRIDGE_API_KEY"), "Set VOCAL_BRIDGE_API_KEY in .env"
assert os.getenv("ANTHROPIC_API_KEY"), "Set ANTHROPIC_API_KEY in .env"
```

Để đơn giản, agent trong notebook chỉ là Claude kèm tool web search. Trong production, agent có thể phức tạp hơn nhiều – LangChain, LangGraph, LlamaIndex, Anthropic Agent SDK, framework nội bộ với RAG, tool, memory, business logic – mẫu tích hợp vẫn áp dụng. **Tất cả những gì Vocal Bridge cần là một hàm nhận chuỗi truy vấn và trả về chuỗi trả lời.**

4.5.2 Xây agent dạng text trước

Nguyên tắc *agent first*: mọi voice agent bắt đầu cuộc đời như một text agent. Xây agent Claude có web search, chứng minh nó chạy tốt với văn bản, *rồi* mới bọc voice. Từ góc nhìn lớp voice, hàm dưới đây chính là toàn bộ agent:

```
import anthropic

claude = anthropic.Anthropic()

def answer(query: str) -> str:
    msg = claude.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=400,
        tools=[{"type": "web_search_20250305", "name": "web_search", "max_uses":
        ↪  2}],
        messages=[{"role": "user", "content": query}],
    )
    parts = [b.text for b in msg.content if getattr(b, "type", None) == "text"]
    return " ".join(parts).strip() or "I don't have an answer for that."

# Quick smoke test
print(answer("Say hi in five words."))
```

4.5.3 Expose agent qua HTTP

Vocal Bridge cần một endpoint để POST truy vấn tới. Trong production, đó chỉ là thêm một route vào backend sẵn có (Flask, FastAPI, LangChain/LangSmith endpoint…). Route đầy đủ chỉ năm dòng:

```
from fastapi import FastAPI

app = FastAPI()

@app.post("/query")
def query(req: dict):
    return {"response": answer(req["query"])}
```

Trong notebook, helper dựng server, chạy health check và trả về URL:

```
server_url = start_query_server(answer, port=8765)
print(f"Query endpoint up: {server_url}")
```

4.5.4 Prompt của lớp voice

Agent Vocal Bridge ở AI Agent mode cần hai artifact. Thứ nhất là system prompt cho lớp voice (`agents/voice-agent/prompt.md`), cho biết việc gì uỷ quyền, việc gì tự trả lời. Ý chính nằm ngay đầu prompt: *bạn là tai và miệng, Claude là bộ não*. Không tự bịa câu trả lời; uỷ quyền mọi câu hỏi thực chất cho endpoint của agent.

Mục quan trọng nhất là chỉ dẫn **bridge line**: ngay khi uỷ quyền một truy vấn, lớp voice phải nói ngay một câu ngắn kiểu "hang on, checking that". Một lượt hỏi đáp với Claude mất 1–6 giây (lâu hơn nếu chạy web search); thiếu bridge line, người dùng ngồi trong im lặng suốt khoảng đó. Vocal Bridge có tự chèn âm đệm tự nhiên khi chờ, nhưng bridge line chính là ranh giới giữa "cảm giác nhanh" và "cảm giác hỏng".

4.5.5 Cấu hình `ai-agent.json`

Artifact thứ hai khai báo thứ gì ở phía bên kia, để Vocal Bridge biết khi nào chuyển tiếp truy vấn, khi nào tự xử lý.

Trường	Ý nghĩa
`enabled`	Bật AI Agent mode.
`description`	Agent giỏi việc gì – căn cứ để Vocal Bridge quyết định uỷ quyền hay tự xử lý small talk. Đây vừa là mô tả vừa là chỉ dẫn cho Vocal Bridge.
`verbatim`	`false`: Vocal Bridge trau chuốt nhẹ câu chữ cho hợp giọng nói (đọc gọn số, bỏ trích dẫn…); `true`: đọc nguyên văn câu trả lời của agent.

Bảng 4.2: Các trường của `ai-agent.json`

Trong bài, `description` mô tả: agent là Claude có web search, biết kiến thức chung và tin tức thời sự; hãy chuyển tiếp mọi câu hỏi thực chất, chỉ tự xử lý small talk. Minh hoạ (dựng lại từ lời giảng, không phải nguyên văn file của khoá học):

```
{
  "enabled": true,
  "description": "Claude with web search. Knows general knowledge and current
  ↪  events. Forward any substantive question; only handle small talk
  ↪  yourself.",
```

```
  "verbatim": false
}
```

4.5.6 Tạo agent

```
agent_id = os.environ.get("VOCAL_BRIDGE_AGENT_ID_L3", "").strip()
if agent_id:
    print(f"Using pre-provisioned agent: {agent_id}")
else:
    result = vb(
        "agent", "create",
        "--name", "lesson-3-voice-agent",
        "--style", "Chatty",
        "--prompt-file", "agents/voice-agent/prompt.md",
        "--ai-agent-file", "agents/voice-agent/ai-agent.json",
        "--background-enabled", "false",
        "--greeting", "",
        "--deploy-targets", "web",
        json_output=True,
    )
    agent_id = result["agent"]["id"]
    append_to_env("VOCAL_BRIDGE_AGENT_ID_L3", agent_id)
```

Hai tham số cần chú ý:

- `--background-enabled false` – AI Agent mode và Background System loại trừ nhau trong Vocal Bridge; tắt Background System lúc tạo để cấu hình AI Agent có hiệu lực.
- `--greeting ""` – để trống lời chào; Claude sẽ sở hữu câu mở đầu, không phải Vocal Bridge.

4.5.7 Widget và chạy thử

Widget kết nối Vocal Bridge, lắng nghe sự kiện `query_agent`, POST tới endpoint cục bộ vừa dựng và trả lời bằng `agent_response`. Mọi thứ khác – transcript, audio, chất keo hội thoại – do Vocal Bridge lo.

```
token = mint_token(agent_id, participant_name="Notebook Learner")
from IPython.display import HTML
HTML(voice_widget(token, query_url=server_url))
```

```
User:  Hey, what was Nvidia's closing price yesterday?
Agent: Let me check that for you. One moment.
Agent: Nvidia's closing price yesterday was $220.61, down 0.769% from the
       previous close of $222.32. The day's low was $217.91 and the high
       was $224.48.
```

Quan sát trong demo: khi chỉ xã giao (“how’s it going”), lớp voice tự trả lời. Khi hỏi giá đóng cửa của Nvidia, nó biết phải uỷ quyền truy vấn cho agent bên dưới – endpoint nhận truy vấn, Claude trả lời, và Vocal Bridge đọc lại câu trả lời đó.

Cuối bài, dừng server để giải phóng cổng:

```
stop_query_server()
```

4.5.8 Hook `useAIAgent`

Toàn bộ phần "cho LLM của tôi một giọng nói" ở phía client là **một React hook** – `useAIAgent({ onQuery })`. Mỗi câu hỏi bằng lời đến dưới dạng `query`; chuỗi nào bạn trả về, Vocal Bridge sẽ nói chuỗi đó:

```
import { useAIAgent } from '@vocalbridgeai/react';

function VoiceLayer() {
  useAIAgent({
    onQuery: async (query) => {
      // VB sends every spoken question here.
      const res = await fetch('/api/query', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ query }),
      });
      const { response } = await res.json();
      return response;  // VB will speak this back to the user.
    },
  });
}
```

Đó là toàn bộ khác biệt phía client giữa text chat và voice chat. Mẫu này chạy với Claude, GPT, Gemini, mô hình fine-tune của bạn, RAG stack, framework agent – bất kỳ thứ gì nhận text vào và trả text ra.

4.6 Tóm tắt: ba mảnh ghép

1. **AI Agent mode của Vocal Bridge** – bật bằng một cờ CLI. Trong production dùng cờ `--ai-agent-enabled true`.
2. **Endpoint của bạn** – bất kỳ HTTP route nào nhận truy vấn và trả về văn bản.
3. **Hai dòng ở client** – `useAIAgent({ onQuery })` trong React, hoặc handler data channel như trong notebook.

Chương 5

Voice như một tool

Đến giờ, mọi cuộc hội thoại voice đều do người dùng khởi xướng – phản hồi một cú chạm, hay bọc quanh agent mà người dùng đang nói chuyện. Chương này (bài 4, notebook `L4.ipynb`) đảo chiều: agent *tự nhấc máy*. Claude được trao tool `make_phone_call`; khi nó quyết định gọi điện là hành động đúng, nó quay một số điện thoại thật, người nhận bắt máy, hai bên trò chuyện, và transcript chảy về theo thời gian thực.

5.1 Voice là một function call

Phần lớn voice agent là *bên nhận*: người dùng bắt đầu cuộc hội thoại. Nhưng nhiều workflow thật cần agent *chủ động*: gọi ứng viên xác nhận lịch rảnh, gọi khách hàng báo tình trạng đơn hàng, hỏi thăm bệnh nhân sau thủ thuật. Cách nhìn mới của chương rất đơn giản:

> **Voice là một function call mà LLM chọn để gọi** – cùng hình dạng với `send_email` hay `query_database`.

Một khi LLM gọi ra ngoài được, điện thoại không còn là kênh chỉ dành cho con người.

5.2 Vì sao tự xây không đơn giản

"LLM gọi điện thoại" nghe như một dòng code. Bên dưới là cả một khối phải nối dây:

Thành phần	Việc phải làm
Nhà cung cấp telephony	Twilio, Plivo, Telnyx…; caller ID đã xác minh; số điện thoại cấp đúng quốc gia.
SIP trunk	Cấu hình outbound, dial plan, định dạng số đúng chuẩn, chính sách xoay vòng caller ID.
Realtime media bridge	Nối audio SIP vào mô hình real-time, chuyển đổi codec.
Agent hội thoại	Một voice agent thực hiện cuộc hội thoại với người ở đầu dây bên kia.
Live observability	Transcript, sự kiện tool truyền ngược về ứng dụng ngay khi cuộc gọi đang diễn ra.
Compliance	Các yêu cầu tuân thủ khi gọi ra ngoài.

Bảng 5.1: Những gì cần nối dây để LLM tự gọi điện

Vocal Bridge gom tất cả về một lệnh CLI: `vb call <số điện thoại>`. Gom gọn được như vậy thì voice mới nằm được trong hộp công cụ của LLM như mọi function khác.

5.3 Các tình huống phổ biến

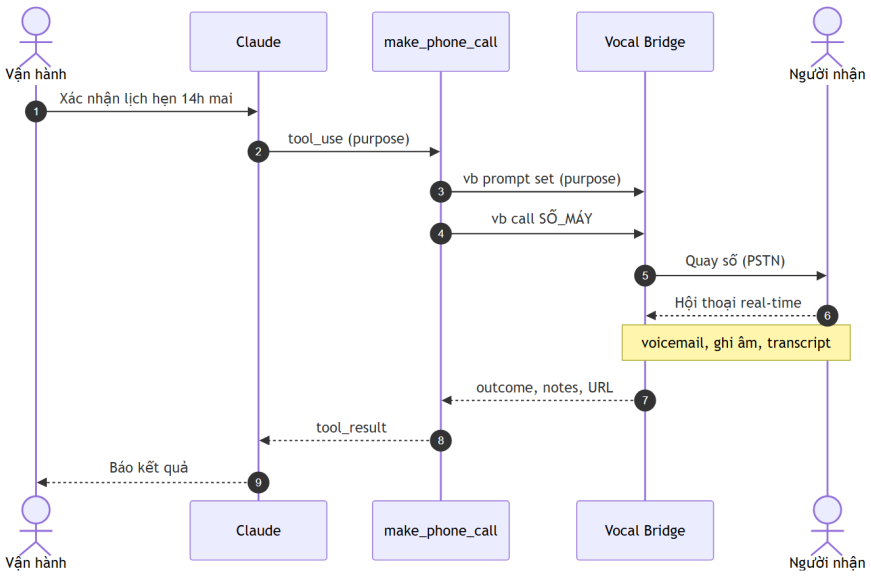
Những mẫu sau lặp lại ở nhiều ngành và đang chạy trong production – có nơi âm thầm tiết kiệm cho cả đội vận hành hàng trăm nghìn giờ:

- **Xác nhận lịch hẹn** (appointment confirmation).
- **Liên hệ ứng viên** (candidate outreach).
- **Cập nhật đơn hàng và vận chuyển**.
- **Sàng lọc khách hàng tiềm năng** (lead qualification) cho bán hàng B2B.
- **Hỏi thăm sau thủ thuật** (post-procedure check-in).
- **Nhắc dịch vụ** – ô tô, ngân hàng: các cuộc gọi thân thiện nhắc gia hạn hoặc bảo dưỡng định kỳ.

5.4 Luồng một cuộc gọi

Ví dụ: một phòng khám nhờ agent chạy bằng Claude xác nhận các lịch hẹn ngày mai (Hình 5.1).

1. Người vận hành giao việc: gọi cho Aditi theo số này để xác nhận lịch hẹn 14h ngày mai.
2. Claude chọn tool `make_phone_call`.
3. Vocal Bridge quay số. Nền tảng xử lý cả voicemail và các tác vụ hậu xử lý sau cuộc gọi.
4. Khi cuộc gọi kết thúc, tool trả về cho Claude một bản tóm tắt có cấu trúc: kết quả (outcome), ghi chú, URL transcript, URL ghi âm.
5. Claude báo kết quả cho người vận hành và chuyển sang lịch hẹn kế tiếp trong danh sách.



Hình 5.1: Sequence diagram – Claude dùng voice như một tool để xác nhận lịch hẹn

LLM ở đây là **bộ điều phối** (orchestrator); voice chỉ là một trong những việc nó làm được.

5.5 An toàn khi gọi ra ngoài

Gọi ra ngoài (outbound) là năng lực mạnh. Vocal Bridge hỗ trợ gọi tới bất kỳ số nào trên thế giới, nhưng để bài học gọn và an toàn, **mọi cuộc gọi của agent trong khoá học đều tới một agent Vocal Bridge khác do khoá học quản lý** (callee agent). Số của callee nằm trong biến môi trường, không nằm trong notebook.

5.6 Thực hành

5.6.1 Chuẩn bị

Cần Vocal Bridge key, Anthropic key, và CLI `vb` đăng nhập đúng tài khoản sở hữu caller agent. Các helper mới của bài:

- `set_caller_purpose(p)` – đưa mục đích của từng cuộc gọi vào prompt của caller agent.
- `place_call(p)` – đặt mục đích rồi quay số tới callee của khoá học; trả về kết quả đã lọc.
- `run_tool_loop()` – chạy trọn một vòng tool-use của Claude.
- `stream_call_transcript()` – stream `vb debug` trong lúc gọi.
- `latest_session()`, `download_recording(sid)` – lấy phiên mới nhất và tải file ghi âm.

5.6.2 Caller agent: base prompt và purpose

Caller là agent *bạn* điều khiển. Mẫu thiết kế giúp "mục đích cuộc gọi" thực sự định hình cuộc hội thoại:

- Caller có một **base prompt** không chứa lý do gọi cụ thể – chỉ có danh tính, phong cách hội thoại và guardrail an toàn. Prompt ghi rõ: mục đích cụ thể của cuộc gọi nằm trong khối `CONTEXT` ở cuối prompt.
- Lý do gọi – **purpose** – được `set_caller_purpose()` chèn vào ngay trước mỗi `vb call`.

Cùng một caller agent, mỗi cuộc gọi một ngữ cảnh runtime khác nhau.

5.6.3 Tạo caller agent

```
agent_id = os.environ.get("VOCAL_BRIDGE_AGENT_ID_L4", "").strip()
if agent_id:
    print(f"Using pre-provisioned caller: {agent_id}")
else:
    result = vb(
        "agent", "create",
        "--name", "lesson-4-tool-caller",
        "--style", "Chatty",
        "--prompt-file", "agents/tool-caller/prompt_base.md",
        "--deploy-targets", "web",
        "--debug-mode", "true",
        json_output=True,
    )
    agent_id = result["agent"]["id"]
    append_to_env("VOCAL_BRIDGE_AGENT_ID_L4", agent_id)
    vb("agent", "use", agent_id)
    vb("config", "set",
       "--outbound-enabled", "true",
```

```
       "--accept-outbound-tos")
    vb("config", "set",
       "--outbound-greeting",
       "Hi — this is an AI voice agent calling about a quick "
       "thing. Got a sec?")

# Pin the caller as the selected agent for the per-call
# `vb prompt set` + `vb call` operations that follow.
vb("agent", "use", agent_id)
```

Ngoài mẫu idempotent quen thuộc, có ba điểm mới:

- `--outbound-enabled true` và `--accept-outbound-tos` – bật gọi ra và chấp nhận điều khoản dịch vụ outbound.
- `--outbound-greeting` – lời chào khi agent gọi ra.
- `--debug-mode true` – cho phép stream sự kiện trong lúc gọi.

Vì sao phải "ghim" agent bằng `vb agent use`: hai bước sau gọi `vb prompt set` và `vb call`, cả hai đều tác động lên agent *đang được chọn* trong ngữ cảnh CLI. Không ghim thì có thể vô tình sửa nhầm agent khác.

5.6.4 Tool schema cho Claude

Định dạng tool chuẩn của Anthropic. Điểm cần để ý: `purpose` là trường **bắt buộc** – đó là ngữ cảnh được chèn vào cuộc gọi. Thiếu nó, callee không biết phải đóng kịch bản nào.

```
TOOLS = [
    {
        "name": "make_phone_call",
        "description": (
            "Place an outbound demo phone call to the course "
            "callee agent. Pass the purpose so the callee "
            "knows what scenario to play."
        ),
        "input_schema": {
            "type": "object",
            "properties": {
                "purpose": {
                    "type": "string",
                    "description": (
                        "What the caller is calling about. "
                        "Drives the entire conversation."
                    ),
                },
                "name": {
                    "type": "string",
                    "description": "Optional friendly callee name",
                },
            },
            "required": ["purpose"],
        },
    }
]
```

5.6.5 Cài đặt tool

```
def make_phone_call(purpose: str, name: str | None = None):
    result = place_call(purpose, name=name)
    print(f"  → call placed, purpose: {purpose}")
    return result
```

Khi Claude chọn `make_phone_call`, hàm gọi `place_call()` trong `helpers.py`, vốn làm ba việc:

1. Đưa purpose vào prompt của caller agent bằng `set_caller_purpose` – chỗ mà tham số purpose "trở thành thật".
2. Quay số tới callee agent do khoá học quản lý; số lấy từ biến môi trường.
3. Trả về cho Claude kết quả đã lọc – chỉ `call_id` và `status`, không để lộ trường nào ở tầng transport.

5.6.6 Để Claude quyết định gọi

```
USER_MESSAGE = (
    "Please place a demo call. Purpose: confirm an appointment "
    "for tomorrow at 2pm."
)

steps = run_tool_loop(
    user_message=USER_MESSAGE,
    tools=TOOLS,
    tool_handlers={"make_phone_call": make_phone_call},
    api_key=os.getenv("ANTHROPIC_API_KEY"),
)
for s in steps:
    print(s)

stream_call_transcript(timeout_s=120)
```

Claude đọc mục đích, chọn `make_phone_call`, gọi với đúng ngữ cảnh, và cuộc gọi đi ra. `stream_call_transcript` dùng `vb debug` để stream mọi sự kiện của agent, callee và tool trong lúc gọi (tối đa 120 giây hoặc tới khi cuộc gọi kết thúc). Có thể đổi purpose để thử các kịch bản khác:

- "Confirm an appointment for tomorrow at 2pm."
- "Check candidate availability for a Thursday interview."
- "Follow up on a delayed shipment, offer alternatives."
- "Just say hi and chat about voice AI for a minute."

5.6.7 Tải và nghe lại ghi âm

Mọi cuộc gọi đều được ghi âm phía server. Lấy phiên hoàn tất gần nhất, tải file ghi âm và phát ngay trong notebook:

```
session = latest_session(status="completed")
session_id = session.get("session_id") or session.get("id")
print(f"Latest session: {session_id}")
print(f"Duration: {session.get('duration_seconds', '?')}s")
```

```
recording_path = download_recording(
    session_id,
    out_path=f"recording_{session_id}.mp3",
)

from IPython.display import Audio
Audio(str(recording_path))
```

Bản ghi trong demo (callee là một agent khác đóng vai người nhận):

```
Callee: Hi there, how can I help? This is an outbound calling.
Caller: Hey, I'm an AI voice agent calling to confirm your appointment
        for tomorrow at 2:00 p.m. Does that time still work for you?
Callee: Yeah, 2:00 p.m. tomorrow works for me, but I plan to arrive
        a little early.
Caller: Arriving 10 minutes early is usually a great idea.
Callee: Got it. I'll do that. Thanks.
```

5.7 Tóm tắt

1. Claude thấy yêu cầu của người dùng, quyết định `make_phone_call` là tool đúng, chọn `purpose`.
2. `place_call()` đưa purpose vào prompt của caller agent và gọi `vb call`.
3. Vocal Bridge quay số tới callee agent; hai voice agent trò chuyện thời gian thực qua mạng điện thoại công cộng (PSTN).
4. Transcript được stream trực tiếp; sau đó tải bản ghi âm và phát lại.

Voice giờ chỉ là một function nữa trong hộp công cụ của Claude – cùng hình dạng với `send_email` hay `query_database`, chỉ khác là nó làm chuông điện thoại reo và có một cuộc trò chuyện.

Chương 6

Đánh giá voice agent

Hệ thống voice *thoái lui trong im lặng* – thường chỉ phát hiện ra khi người dùng phàn nàn. Các chương trước là về *xây*; chương này (bài 5, notebook `L5.ipynb`) là về *không phát hành theo cảm tính* (not shipping by vibes). Công cụ chính là evaluator đa phương thức có sẵn của Vocal Bridge: nó chấm điểm cuộc gọi, đề xuất sửa prompt, và giúp bắt lỗi trước khi lỗi tới production.

6.1 Eval và Eval-Driven Development

Một **eval** là bài kiểm tra lặp lại được, chấm điểm hệ thống AI theo đúng những kết quả bạn quan tâm. Hãy nghĩ nó như unit test, nhưng cho một hệ thống không tất định (non-deterministic) mà bạn không thể so sánh bằng diff. Mọi hệ thống AI được phát hành đều cần eval, vì:

- Một chỉnh sửa prompt trông sạch sẽ có thể âm thầm làm hỏng ba edge case.
- Đổi mô hình làm thay đổi hành vi theo cách không integration test nào bắt được.
- Không có ground truth, bạn phát hiện regression từ khách hàng – nơi tệ nhất để phát hiện.
- Không có điểm số, bạn không biết một thay đổi làm hệ thống tiến lên hay lùi lại.

Eval-Driven Development là xây và lặp trên hệ thống lấy điểm eval làm tín hiệu chính. Đó là kỷ luật biến một demo thành một sản phẩm.

6.2 Vì sao voice eval khó hơn text eval

Text eval chỉ so chuỗi. Voice không cho bạn sự xa xỉ đó: có audio, thời điểm, tone, nhiều lượt, một agent có thể ngắt lời hoặc bị ngắt lời – và không thứ nào hiện ra trong text diff, kể cả trong transcript. Sáu điều làm voice eval khó hơn:

1. **Tín hiệu đa phương thức** – một lượt nói mang text, audio, nhịp, prosody. Chấm một chiều là bỏ sót các chiều khác.
2. **Regression im lặng** – một giọng TTS mới có thể nghe như robot ở vài cách diễn đạt cụ thể, trong khi transcript giống hệt.
3. **Turn-taking quan trọng** – agent có chờ người dùng nói xong không? Có ngắt lời đúng lúc không? Chỉ text thì không trả lời được.
4. **Cuộc gọi khó phát lại** – không thể chạy lại một cuộc gọi trực tiếp một cách tất định; bản ghi âm cộng log có cấu trúc là ground truth duy nhất.

5. **Tool call xảy ra ngay giữa hội thoại** – agent có gọi đúng function, đúng lúc, đúng tham số không?
6. **Độ trễ là một phần trải nghiệm** – một câu trả lời đúng nhưng trễ bốn giây cũng là câu trả lời sai.

Chỉ chấm phần text, bạn sẽ bỏ lỡ phần lớn những gì thực sự hỏng.

6.3 Hai lớp đánh giá

6.3.1 Lớp 1 – chỉ số định lượng cứng

Những thứ đo được một cách tất định: dựng baseline từ chúng và cho build thất bại khi có regression.

Chỉ số	Đo cái gì
WER (Word Error Rate)	Độ chính xác phiên âm.
MOS (Mean Opinion Score)	Chất lượng giọng tổng hợp.
TTFB (Time to First Byte)	Độ trễ tới token hoặc gói audio đầu tiên.
Turn duration	Thời lượng một lượt, đầu cuối.
Tool-call accuracy	Agent có gọi đúng function với đúng tham số không, và có kích hoạt tool call khi cần không.
Completion rate	Tỉ lệ hoàn thành.

Bảng 6.1: Các chỉ số định lượng cho voice agent

6.3.2 Lớp 2 – multimodal LLM làm giám khảo

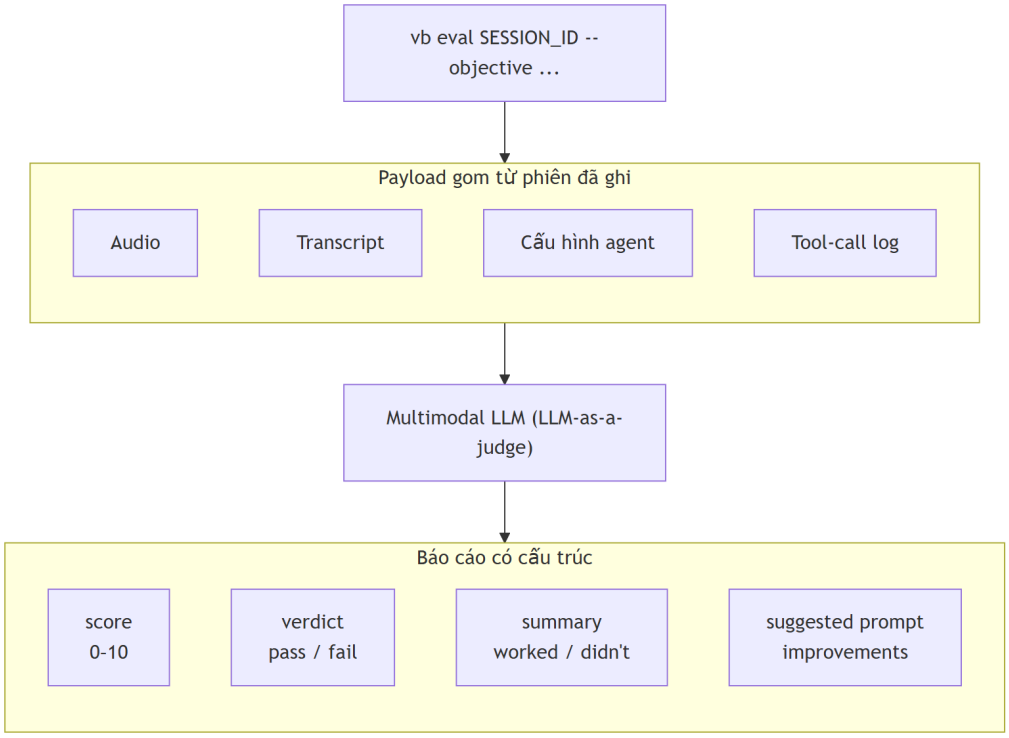
Dùng một LLM đa phương thức làm giám khảo (LLM-as-a-judge) cho những điều định tính mà chỉ số không nhìn thấy:

- Tone của agent có hợp hoàn cảnh không?
- Turn-taking có tự nhiên, không có khoảng lặng gượng gạo?
- Cuộc gọi có thực sự đạt mục tiêu?
- Agent có xử lý khéo khi người dùng đi lệch kịch bản?
- Agent phục hồi sau bị ngắt lời hoặc gặp lỗi tốt đến đâu?
- Và quan trọng: các **chỉnh sửa prompt cụ thể** mà giám khảo đề xuất cho lần sau.

Chỉ số định lượng bắt cái khách quan; giám khảo bắt sự trôi dạt định tính vốn vô hình. Kết hợp cả hai – đặc biệt ở quy mô lớn – mới làm voice eval khả thi.

6.4 Evaluator `vb eval`

Vocal Bridge gói toàn bộ quá trình vào một lệnh CLI: truyền `session_id` của cuộc gọi và **objective**. Lệnh gom bản ghi âm, transcript, cấu hình agent và tool-call log thành payload gửi cho giám khảo đa phương thức, rồi nhận về một báo cáo có cấu trúc có thể nối vào vòng CI/CD (Hình 6.1).



Hình 6.1: Luồng của `vb eval`: từ phiên đã ghi đến báo cáo có cấu trúc

Một báo cáo mẫu trong bài giảng:

```
Objective : Confirm appointment for tomorrow at 2pm
Outcome   : Confirmed
Score     : x/10
Breakdown : conversation quality, turn-taking (natural / awkward),
            caller satisfaction, ... (tiêu chí định nghĩa trong scenario)
Suggestion: Soften the opening greeting -- the current phrasing reads as
            scripted. Try: "Hi! Just checking in on your 2pm tomorrow..."
```

Phần giá trị nhất – giúp khép vòng lặp – là gợi ý sửa cấu hình agent.

6.5 Thực hành

Notebook cần ít nhất một cuộc gọi đã hoàn tất có ghi âm; nguồn dễ nhất là cuộc gọi ở Chương 5. Bài chỉ dùng hai helper: `vb` và `eval_session` – một wrapper một dòng quanh `vb eval`. Đó là toàn bộ bề mặt API của eval.

6.5.1 Tìm phiên để chấm

```
recent = vb(
    "logs", "list", "-n", "5",
    "--status", "completed",
    json_output=True,
)
sessions = recent.get("logs") or recent.get("sessions") or []
for r in sessions:
    sid = r.get("session_id") or r.get("id")
    started = r.get("started_at", "?")
```

```
    dur = r.get("duration_seconds", "?")
    print(f"  {sid}  {started}  {dur}s")
```

Nếu đã chạy cuộc gọi ở bài 4, `session_id` của nó nằm đầu danh sách.

6.5.2 Chấm một cuộc gọi theo objective

Objective là điều agent lẽ ra phải đạt trong cuộc gọi – thứ làm cho eval có ý nghĩa. *Không có objective, LLM chỉ đang chấm "cảm giác"*; có objective, nó nói được cuộc gọi thành công hay không. Bên dưới, `eval_session()` chỉ là một lệnh:

```
vb eval <session_id> \
   --objective "Confirm the appointment for tomorrow at 2pm" \
   --json
```

```
SESSION_ID = "INPUT A SESSION ID FROM THE ABOVE CELL OUTPUT HERE"
OBJECTIVE = (
    "INPUT THE OBJECTIVE HERE"
)

report = eval_session(SESSION_ID, objective=OBJECTIVE)
print(json.dumps(report, indent=2)[:2000])
```

Trong demo, objective là: caller thực hiện một cuộc gọi demo ra ngoài và có một cuộc trò chuyện ngắn, đúng chủ đề với callee. Lệnh mất vài giây vì giám khảo đa phương thức phải xử lý audio, cấu hình, transcript và tool log.

6.5.3 Đọc báo cáo

```
print(f"Score:   {report['result'].get('score', '–')}")
print(f"Verdict: {report['result'].get('verdict', '–')}")
print()
print("Suggestions for prompt improvements:")
print(report['result'].get("suggested_prompt_improvements"))
```

Trường	Ý nghĩa
`session_id`, `objective`	Phiên được chấm và mục tiêu đã truyền vào.
`result.score`	Điểm chất lượng 0–10.
`result.verdict`	Quyết định nhị phân: đạt / không đạt objective.
`result.summary`	Nhận xét định tính về cách agent thể hiện, kèm điều làm tốt và chưa tốt.
`result.suggested_prompt_improvements`	Gợi ý sửa prompt và cấu hình để đạt objective, tiến gần điểm tuyệt đối.

Bảng 6.2: Các trường chính của báo cáo eval

Trong demo, cuộc gọi đạt 10/10, verdict *pass*, mọi thứ đều ổn (prompt khá đơn giản), và phần gợi ý ghi rằng prompt đã rất hiệu quả cho kịch bản này, không cần sửa. Hai thứ bạn sẽ thực sự hành động theo là **score** và **suggestions**: nếu cuộc gọi trượt objective, mô hình chỉ ra chính xác chỗ sai (ví dụ "agent didn't confirm the purpose before hanging up") – đó chính là prompt diff của bạn.

6.5.4 Xây một eval suite nhỏ

Một cuộc gọi chưa phải là suite. Chọn 3–5 phiên bao phủ các kiểu lỗi bạn quan tâm – voicemail, callee bận, chỉ dẫn mơ hồ, phục hồi sau khi bị ngắt lời – mỗi phiên kèm objective và **scenario**. Cùng một agent, cùng objective, nhưng khác tình huống:

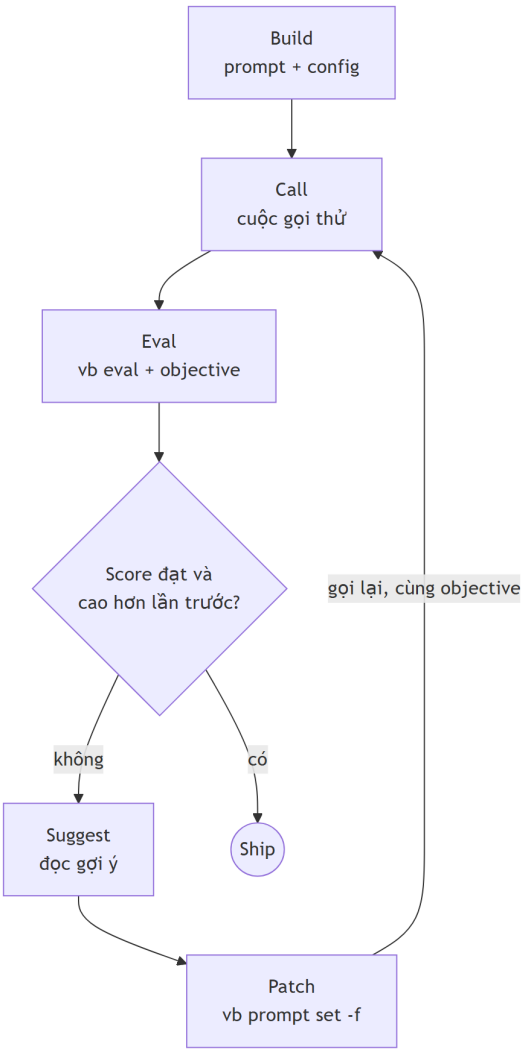
```
SUITE = [
    # (session_id, objective, scenario)
    # ("abc123…", "Confirm appointment for tomorrow at 2pm.", "Callee
    ↪  confirms."),
    # ("def456…", "Confirm appointment for tomorrow at 2pm.", "Callee asks to
    ↪  reschedule."),
]

results = []
for session_id, objective, scenario in SUITE:
    rep = eval_session(session_id, objective=objective, scenario=scenario)
    results.append({
        "session": session_id[:8],
        "score":   rep['result'].get("score"),
        "verdict": rep['result'].get("verdict"),
    })

import pprint
pprint.pp(results)
```

Bằng cách này có thể đánh giá agent toàn diện, ở quy mô lớn, trước khi lên production.

6.5.5 Khép vòng lặp



Hình 6.2: Vòng lặp eval-driven development

Kỷ luật nằm ở *vòng lặp*, không ở một con số đơn lẻ: build → call → eval → đọc gợi ý → patch prompt → call → eval lại. Chọn phiên điểm thấp nhất, đọc gợi ý, sửa prompt bằng `vb prompt set -f <updated.md>`, gọi một cuộc thử mới rồi chấm lại. Notebook để đoạn dưới ở dạng comment để người học tự điều chỉnh theo phiên của mình:

```
# 1. Read the current prompt
current = vb("prompt", "show")

# 2. Edit it
new_prompt = current.replace("OLD_LINE", "NEW_LINE")
Path("/tmp/updated_prompt.md").write_text(new_prompt)

# 3. Push it
print(vb("prompt", "set", "-f", "/tmp/updated_prompt.md"))

# 4. Place another test call (Lesson 4 pattern), capture its new
#    session_id, then re-eval against the same objective.
report2 = eval_session(NEW_SESSION_ID, objective=OBJECTIVE)
```

```
print("before:", report.get("score"), "after:", report2.get("score"))
```

6.6 Ràng buộc cần giữ

- Luôn chấm theo **objective** rõ ràng; không có objective thì điểm số vô nghĩa.
- **Một regression chỉ được coi là đã sửa khi một cuộc gọi mới, cùng objective, đạt điểm cao hơn.** Eval suite là thứ giữ cho bạn trung thực.
- Đo cả lớp định lượng (WER, TTFB, tool-call accuracy…) lẫn lớp định tính (giám khảo đa phương thức); chỉ một lớp là bỏ sót lỗi.
- Bản ghi âm và log có cấu trúc là ground truth – giữ chúng cho mọi phiên cần đánh giá.

Đến đây khoá học đã đi qua cả ba bề mặt của Vocal Bridge – voice trong ứng dụng, voice cho agent có sẵn, voice như một tool – và cài vòng lặp eval-driven development lên trên. Bước tự nhiên tiếp theo là làm cứng cho production: xác thực, giám sát, tinh chỉnh độ trễ – chủ đề của Chương 7. Tài liệu Vocal Bridge tại `https://vocalbridgeai.com/docs/overview` tiếp nối đúng từ điểm này.

Chương 7

Voice agent trong production

Bài cuối của khoá học là cuộc trò chuyện giữa Ashwyn Sharma và **Scott Johnston** – cựu CEO Docker, thành viên hội đồng quản trị Vocal Bridge – về những gì thực sự cần để đi từ demo đến voice trong production. Phần lớn khoá học về voice AI dạy cách dựng một demo chạy được; rất ít khoá chuẩn bị cho những gì xảy ra *sau khi* demo được trao vào tay người dùng. Chương này tóm lược các ý chính của cuộc trò chuyện.

7.1 Bài học từ Docker

Scott đã hơn mười năm ở Docker và đưa nền tảng này tới hơn 26 triệu lập trình viên. Ngày nay container trông như "ống nước" đã giải xong, nhưng mười năm trước, triển khai phần mềm thực sự khó. Điều gì đã thay đổi?

- **Không phải công nghệ nền được phát minh** – Linux namespaces và cgroups đã có từ nhiều năm trước đó.
- **Mà là con đường được rút ngắn** – từ "tôi có một ý tưởng" tới "nó đang chạy trong production" đủ ngắn để *một* lập trình viên tự đi hết.
- Khi con đường ngắn lại, số người có thể phát hành sản phẩm tăng lên nhiều bậc độ lớn, và *loại* sản phẩm họ làm ra cũng thay đổi, vì chi phí để thử một điều mới trở nên rất thấp.

Hình dạng của bài toán "khoảng cách tới production" lặp lại đáng kinh ngạc qua mỗi thế hệ công nghệ, dù công nghệ mỗi lần một khác.

7.2 Áp vào voice

Các mô hình nền đã tiến bộ vượt bậc trong vài năm qua, nhưng con đường từ "tôi muốn ứng dụng của mình nói chuyện với người dùng" tới "một triệu người dùng nó một cách tin cậy" vẫn dài, và vẫn chủ yếu do chuyên gia đi. Câu hỏi thực sự là *ai được xây thứ tiếp theo*: nếu chỉ mười đội trên thế giới đủ kiên nhẫn để tự nối mọi thứ, ta có mười sản phẩm voice. Nếu một triệu lập trình viên làm được trong một cuối tuần, ta có một hạng mục sản phẩm hoàn toàn khác.

7.3 "Production" nghĩa là gì

Production không phải là mô hình trả lời đúng trong một notebook. Production là: hai giờ sáng, một người dùng ở São Paulo đang nói chuyện với ứng dụng của bạn; hệ thống phải phản hồi trong **dưới 600 ms**, đúng phương ngữ, đúng giọng vùng miền đó, trong khi gọi payments API ngay giữa câu. Nếu bất kỳ phần nào hỏng, bạn cần biết **trong vòng một giờ**, không phải một tuần sau từ một dòng tweet giận dữ.

Từ vựng ở đây giống mọi hệ thống high availability: **latency budget**, **observability**, **evaluation**, **graceful fallback**. Không vấn đề nào riêng của voice – chúng là vấn đề của mọi hệ thống phân tán. Thứ *riêng* của voice là con người cực kỳ khó chịu với lỗi giọng nói:

- Chat agent dừng hai giây – bạn chờ.
- Voice agent dừng hai giây – bạn nghĩ nó đã "chết" và cúp máy.

Ngưỡng chịu đựng thấp hơn nhiều, nên rào cản kỹ thuật phải cao hơn nhiều. Chuẩn cho voice trong production gần với *hệ thống giao dịch thời gian thực* hơn là một web app thông thường; không thể hoãn phần khó lại sau.

7.4 Hai bề mặt mới ngoài contact center

Nơi duy nhất voice agent đang được triển khai nghiêm túc hiện nay là contact center – kinh tế rõ ràng, workflow bị ràng buộc, có người mua. Nhưng đó chỉ là một lát mỏng. Hai bề mặt lớn hơn nhiều đang mở ra song song:

7.4.1 Voice như một tính năng bên trong ứng dụng

Dashboard logistics cho tài xế không thể rời mắt khỏi đường, trợ lý lập trình mà bạn có thể cùng nói chuyện để gỡ một bug, hệ thống bệnh án điện tử mà bác sĩ đọc để chép, ứng dụng học cho trẻ chưa biết đọc. Ở mỗi trường hợp, voice *không phải* là sản phẩm – nó là một tính năng của sản phẩm. Đây là bài toán thiết kế hoàn toàn khác với contact center: lớp voice phải sống cạnh trạng thái sẵn có của ứng dụng, gọi *cùng những tool* mà các nút bấm của ứng dụng đang gọi, và tôn trọng thứ người dùng đang nhìn trên màn hình. Stack xây cho call center chưa bao giờ được thiết kế cho điều đó (Chương 3).

7.4.2 Voice cho AI agent

Bề mặt thứ hai gần như chưa tồn tại hai, ba năm trước. Hàng triệu lập trình viên đã xây AI agent: agent đọc codebase, agent phân loại ticket, agent chạy phân tích, agent vận hành bên trong các workflow lớn hơn. Hầu hết chúng hiện chỉ có text, và voice là giao diện tự nhiên tiếp theo cho phần lớn trong số đó. Điểm thú vị: phần việc khó – suy luận, dùng tool, memory – đã xong; thứ còn thiếu chỉ là *phương thức*. Người đã xây agent không nên phải xây lại từ đầu chỉ để cho nó một giọng nói (Chương 4).

Vì thế bài toán tích hợp không chỉ còn là *voice + app*, mà còn là *voice + agent*. Và mười triệu nhà phát triển voice tiếp theo sẽ không tự coi mình là "voice developer": họ là người năm ngoái phát hành một web app, năm nay phát hành một agent. Voice chỉ là một bề mặt nữa họ dùng tới khi đó là lựa chọn đúng.

7.5 Những khó khăn đội ngũ thực tế hay gặp

Khó khăn	Bản chất
Mô hình tích hợp	Phần lớn voice stack giả định agent sở hữu toàn bộ cuộc hội thoại. Nhưng ứng dụng thật có business logic, xác thực, database, tool sẵn có. Voice agent phải là *một thành viên* của hệ thống đó, không phải toàn bộ hệ thống. Giống thời đầu của container: người ta tưởng container phải là cả một VM; ngành mất vài năm mới thấm rằng container chỉ là một mảnh của hệ thống lớn hơn, và giá trị đến từ việc nó kết hợp gọn gàng với mọi thứ xung quanh.
Đánh giá	Voice chưa có thứ tương đương unit test, vì đầu ra là dạng sóng âm thanh còn đầu vào là một cuộc hội thoại. Các đội phát hành "mù", nghe về một cuộc gọi tệ từ người dùng, và không có cách hệ thống nào để tái hiện, kiểm thử regression hay chứng minh bản sửa có hiệu quả (Chương 6).

Bảng 7.1: Hai khó khăn chính khi đưa voice agent vào production

Xây vòng lặp **deploy – observe – evaluate – iterate** chính là nơi phần lớn công việc production thực sự nằm.

7.6 Thông điệp chính

- **Mô hình không còn là nút thắt cổ chai.** Nút thắt là "phần giữa nhàm chán" (the boring middle): deploy, observe, evaluate, iterate. Thế hệ sản phẩm voice tiếp theo sẽ thắng hay thua ở đó, và đó cũng là cơ hội cho lập trình viên.
- **Các bài toán chưa giải là bài toán kỹ thuật phần mềm**, không phải bài toán mô hình. Người học khoá này đã đi qua vòng lặp deploy – observe – evaluate – iterate với Vocal Bridge – chính kinh nghiệm đó giúp sẵn sàng cho triển khai voice ở cấp production.
- **Lịch sử lặp lại**: mỗi khi phần giữa nhàm chán trở nên dễ hơn nhiều – với container, web hosting, mobile – một làn sóng sản phẩm mới xuất hiện từ những người vừa được trao khả năng xây chúng. Voice là cái tên tiếp theo trong danh sách đó.

Phụ lục A

Thuật ngữ và tra cứu nhanh

Phụ lục gom các thuật ngữ cốt lõi của khoá học (theo bảng thuật ngữ đi kèm khoá học), các lệnh CLI `vb` và các helper dùng trong notebook để tra cứu nhanh.

A.1 Bảng thuật ngữ

A.1.1 Từ viết tắt và chỉ số

Các khối xây dựng kỹ thuật và tiêu chuẩn đo hiệu năng hệ thống voice.

Thuật ngữ	Giải thích
ASR / STT (Speech-to-Text)	Bước đầu vào: phiên âm audio thành chuỗi văn bản. Còn gọi là Automatic Speech Recognition (nhận dạng tiếng nói tự động).
GUI (Graphical User Interface)	Giao diện đồ hoạ truyền thống (app, website), thường bổ trợ cho voice trong ứng dụng đa phương thức.
IVR (Interactive Voice Response)	Tổng đài tự động truyền thống, tương tác với người gọi qua giọng nói và phím bấm (DTMF).
LLM (Large Language Model)	"Bộ não" của agent: engine suy luận xử lý prompt, quản lý tool và sinh câu trả lời.
MOS (Mean Opinion Score)	Điểm 1–5 thể hiện chất lượng và độ tự nhiên của giọng tổng hợp theo cảm nhận của con người.
SIP (Session Initiation Protocol)	Giao thức báo hiệu cốt lõi để định tuyến, thiết lập và quản lý cuộc gọi thoại thời gian thực.
TTFB (Time to First Byte)	Chỉ số độ trễ then chốt: thời gian từ lúc người dùng nói xong tới khi hệ thống sinh token hoặc gói audio đầu tiên.
TTS (Text-to-Speech)	Engine tổng hợp, chuyển văn bản thành giọng nói nhân tạo.
VAD (Voice Activity Detection)	Xác định audio đầu vào có chứa giọng người đang nói hay chỉ là tiếng ồn nền.
WER (Word Error Rate)	Chỉ số tất định đo độ chính xác phiên âm bằng cách so với transcript chuẩn (ground truth).

Bảng A.1: Thuật ngữ – từ viết tắt và chỉ số

A.1.2 Kiến trúc

Cách hệ thống được tổ chức và cách dữ liệu đi giữa các thành phần.

Thuật ngữ	Giải thích
Cascaded Stack (“Sandwich”)	Pipeline voice truyền thống nối các mô hình riêng theo chuỗi (STT → LLM → TTS). Dễ gỡ lỗi nhưng độ trễ cao (1–3 giây) và mất sắc thái cảm xúc.
Concierge Architecture	Kiến trúc lai của Vocal Bridge: foreground agent nhanh lo luồng thời gian thực (filler, turn-taking), uỷ quyền tác vụ phức tạp cho background LLM agent để suy luận sâu và dùng tool.
Orchestrator	Lớp hệ thống điều phối tương tác phức tạp: quản lý trạng thái, phối hợp thời điểm giữa các thành phần, xử lý ngắt lời.
Real-time Stack (Voice-to-Voice)	Một mô hình đầu cuối xử lý trực tiếp audio. Độ trễ cực thấp (200–500 ms) nhưng khó kiểm soát bằng kỹ thuật chuyên ngành.
Telephony Bridge	Hạ tầng (SIP, PSTN, hoặc nhà cung cấp như Twilio) nối voice agent số với đường điện thoại truyền thống.
Vocal Bridge (VB)	Nền tảng cung cấp “bộ não voice-to-voice” đứng trước agent có sẵn để quản lý hội thoại, turn-taking và paralinguistics.
WebRTC (Web Real-Time Communication)	Giao thức truyền thông thời gian thực dùng cho data channel hai chiều mang audio và trạng thái ứng dụng giữa agent và client.

Bảng A.2: Thuật ngữ – kiến trúc

A.1.3 Trải nghiệm hội thoại

Các yếu tố quyết định một tương tác voice “cảm thấy” thế nào với người dùng.

Thuật ngữ	Giải thích
Barging / Interruption	Khả năng xử lý khéo khi người dùng nói chen ngang giữa câu: agent dừng nói và thích nghi với đầu vào mới.
Bridge Line	Câu nói ngắn (vd. “Let me check on that…”) foreground agent dùng để lấp khoảng lặng trong lúc uỷ quyền truy vấn phức tạp cho background agent.
Endpointing	Xác định chính xác khi nào người dùng nói xong để agent biết lúc phản hồi.
Latency	Độ trễ giữa đầu vào của người dùng và phản hồi của agent; giảm độ trễ là điều kiện để hội thoại tự nhiên.
Multimodal	Giao diện dùng nhiều phương thức tương tác, vd. voice agent đồng thời cập nhật GUI.
Paralinguistics / Prosody	Yếu tố phi ngôn từ của lời nói – tone, nhịp, cao độ, cảm xúc – mang ý định và ngữ cảnh ngoài nghĩa đen của văn bản.
Turn-taking	Thời điểm và logic quyết định khi nào agent nên nói, chờ, hay lắng nghe.

Bảng A.3: Thuật ngữ – trải nghiệm hội thoại

A.1.4 Phát triển và phương pháp

Thực hành kỹ thuật để xây, kiểm thử và triển khai hệ thống AI không tất định.

Thuật ngữ	Giải thích
Client Actions	Mẫu dùng kênh giao tiếp hai chiều (WebRTC) để giữ voice agent và UI ứng dụng đồng bộ hoàn toàn.
Eval-Driven Development	Kỷ luật kỹ thuật lấy điểm đánh giá tự động làm tín hiệu chính để lặp trên hệ thống AI, lấp khoảng cách giữa “demo” và “production”.
Evals (Evaluations)	Bài kiểm tra có hệ thống, lặp lại được, chấm kết quả của hệ thống AI theo mục tiêu hoặc ground truth cụ thể.
Idempotent Pattern	Cách viết code để khối setup chạy nhiều lần vẫn an toàn: dùng lại ID tài nguyên đã có thay vì tạo agent trùng lặp.
Multimodal Judge	LLM dùng để chấm các khía cạnh định tính của tương tác voice, như tone có hợp hoàn cảnh hay hội thoại có tự nhiên.
RAG (Retrieval-Augmented Generation)	Kỹ thuật cung cấp tri thức chuyên ngành hoặc memory cho LLM bằng cách truy xuất thông tin từ tài liệu bên ngoài.
Telemetry	Thu thập và phân tích dữ liệu thời gian thực (độ trễ, tỉ lệ lỗi, chi phí) để giám sát hiệu năng hệ thống trong production.

Bảng A.4: Thuật ngữ – phát triển và phương pháp

A.2 Lệnh CLI vb và SDK

Lệnh	Tác dụng	Chương
`vb auth login`	Đăng nhập CLI (khi chạy ngoài môi trường khoá học).	1
`vb agent create --name ... --style Chatty --prompt-file ...`	Tạo agent. Cờ tuỳ chọn: `--client-actions-file` (Client Actions), `--ai-agent-file` (AI Agent mode), `--deploy-targets`, `--debug-mode`…	3–5
`vb agent use <agent_id>`	Chọn (ghim) agent làm ngữ cảnh cho các lệnh sau.	3–5
`vb config set --outbound-enabled true --accept-outbound-tos`	Bật gọi ra và chấp nhận điều khoản outbound.	5
`vb config set --outbound-greeting "..."`	Đặt lời chào khi agent gọi ra.	5
`vb prompt show`	Xem prompt của agent đang chọn.	6
`vb prompt set -f <file.md>`	Cập nhật prompt của agent đang chọn.	5, 6
`vb call <số điện thoại>`	Agent đang chọn gọi ra một số điện thoại.	5
`vb debug`	Stream sự kiện của agent, callee, tool trong lúc gọi.	5
`vb logs --limit 1`	Lấy phiên gần nhất.	3, 4
`vb logs list -n 5 --status completed`	Liệt kê các phiên đã hoàn tất.	6
`vb logs show <session_id>`	Xem log có cấu trúc của một phiên: lượt nói, action, sự kiện.	3, 4
`vb eval <session_id> --objective "..." --json`	Chấm một phiên bằng evaluator đa phương thức.	6

Bảng A.5: Các lệnh CLI `vb` dùng trong khoá học

API / SDK	Tác dụng	Chương
`POST /api/v1/token`	Backend mint voice session token ngắn hạn (header `X-API-Key`, `X-Agent-Id`).	3
`VocalBridgeProvider`	Component React voice hoá một ứng dụng.	2
`useAgentActions()`	Client Actions hai chiều: `onAction` nhận, `sendAction` gửi.	3
`useAIAgent({ onQuery })`	Nhận mỗi câu hỏi bằng lời, trả về chuỗi để Vocal Bridge nói.	4

Bảng A.6: REST API và React SDK (`@vocalbridgeai/react`)

A.3 Các helper trong notebook

Các helper trong `helpers.py` chỉ gói phần boilerplate; mỗi helper là vài dòng Python hoặc React mà trong ứng dụng thật bạn sẽ tự viết.

Helper	Tác dụng	Bài
`load_env()`, `append_to_env()`	Nạp `.env`; ghi ID agent mới tạo vào `.env`.	2–5
`mint_token(agent_id)`	Lấy voice session token – một `requests.post()` tới `/api/v1/token`.	2, 3
`vb(*args)`	Chạy lệnh CLI `vb` và parse kết quả – `subprocess.run(["vb", …])` + JSON.	2–5
`voice_widget(token, [query_url])`	Render widget React trong notebook, dùng hook của `@vocalbridgeai/react`.	2, 3
`start_query_server(handler)`, `stop_query_server()`	Dựng / dừng server FastAPI nhỏ trong thread nền để widget POST truy vấn tới.	3
`run_tool_loop()`	Chạy trọn một vòng tool-use của Claude.	4
`set_caller_purpose(p)`	Đưa mục đích cuộc gọi vào prompt của caller agent.	4
`place_call(p)`	Đặt purpose, quay số tới callee của khoá học, trả kết quả đã lọc.	4
`stream_call_transcript()`	Stream `vb debug` trong lúc gọi.	4
`latest_session()`, `download_recording(sid)`	Lấy phiên mới nhất; tải ghi âm để phát lại.	4
`eval_session(session_id, objective)`	Chạy `vb eval` và trả về báo cáo dạng dict.	5

Bảng A.7: Các helper trong notebook của khoá học